ARTIGO DE REVISÃO DIDÁTICO

Do sinal à rede neural: uma revisão didática da classificação de glitches do LIGO com aprendizado profundo e do desafio dos dados reais

From signal to neural network: a didactic review of LIGO glitch classification with deep learning and the challenge of real data

Projeto 3 – Ondas Gravitacionais · PIC Astronomia

Versão final de 30 de outubro de 2026 · código, dados derivados e relatórios em github.com/jean5429/gw-pic-2026

Resumo. Este artigo de revisão didático percorre, passo a passo, o caminho entre o sinal bruto de um detector de ondas gravitacionais e a decisão de uma rede neural. O fio condutor é um estudo de caso: o Projeto 3 do PIC Astronomia, que treinou uma ResNet-18 para classificar glitches do LIGO a partir das imagens do Gravity Spy e a aplicou aos eventos do catálogo GWTC-4. A revisão segue a ordem em que o projeto foi construído. Primeiro, os dados e o condicionamento do sinal: densidade espectral do ruído, branqueamento, filtros e a transformada Q. Depois, a validação física com o GW150914: alinhamento entre detectores, massa de chirp pela trilha tempo-frequência e filtro casado. Por fim, as ferramentas de aprendizado de máquina e aprendizado profundo: transferência de aprendizado, entropia cruzada ponderada, AdamW, ensembles, calibração por temperatura, detecção fora da distribuição por energia e distância de Mahalanobis, e Grad-CAM. Cada etapa traz as fórmulas usadas no código. No estudo de caso, a rede atinge macro-F1 de 0,945 nas imagens do Gravity Spy, mas as 475 imagens dos eventos são rejeitadas como desconhecidas. Testes de fechamento, em que glitches já rotulados passam pela cadeia de imagens do próprio projeto, mostram que a forma de gerar as imagens explica boa parte dessa queda: a acurácia cai para 0,72–0,75, sobretudo nas classes de longa duração. Discutimos as lições metodológicas e as perspectivas.

Palavras-chave: ondas gravitacionais; LIGO; glitches; Gravity Spy; transformada Q; redes neurais convolucionais; detecção fora da distribuição; mudança de domínio.

Abstract. This didactic review follows, step by step, the path from the raw output of a gravitational-wave detector to the decision of a neural network. The guiding thread is a case study: Projeto 3 of PIC Astronomia, which trained a ResNet-18 to classify LIGO glitches on Gravity Spy images and applied it to the events of the GWTC-4 catalog. We review, with the formulas used in the code, spectral estimation, whitening, filtering and the Q transform; the physical checks performed on GW150914 (inter-detector alignment, chirp mass from the time-frequency ridge and matched filtering); and the machine-learning and deep-learning tools (transfer learning, weighted cross-entropy, AdamW, ensembles, temperature scaling, energy and Mahalanobis out-of-distribution scores, and Grad-CAM). The network reaches a macro-F1 of 0.945 on Gravity Spy images, but all 475 event images are rejected as out of distribution. Closure tests, in which labeled glitches are re-rendered by the project's own pipeline, show that the image-generation chain accounts for a large part of this gap: accuracy drops to 0.72–0.75, mostly for long-duration classes. We discuss the methodological lessons and perspectives.

Keywords: gravitational waves; LIGO; glitches; Gravity Spy; Q transform; convolutional neural networks; out-of-distribution detection; domain shift.

> **Como ler este artigo.** Cada seção apresenta primeiro o conceito e a literatura, depois a aplicação no estudo de caso, e termina com um quadro **Em poucas palavras**. As fórmulas são numeradas e explicadas símbolo por símbolo; quem quiser só a intuição pode pular as equações sem perder o fio.
>
> Os termos técnicos estão no glossário (Apêndice A). As classes do Gravity Spy aparecem com os nomes do conjunto de dados original (Blip, Koi_Fish, Whistle etc.).

1. Introdução

A detecção do GW150914, em 2015, abriu a astronomia de ondas gravitacionais [1]. Desde então, os detectores LIGO, Virgo e KAGRA registraram centenas de candidatos, hoje reunidos no catálogo GWTC [2]. Medir esses sinais exige separar uma deformação do espaço da ordem de 10^{-21} de um ruído instrumental muito maior e, em especial, de transientes de ruído — os *glitches* — que podem imitar ou esconder um evento astrofísico [3]. O aprendizado de máquina tornou-se uma ferramenta central nessa tarefa, tanto para classificar glitches quanto para buscar sinais [4].

Este artigo tem dois objetivos. O primeiro é **didático**: apresentar, numa sequência única e com as fórmulas completas, as etapas que levam do dado bruto à decisão de uma rede neural — estimativa do ruído, branqueamento, filtragem, transformada Q, verificações físicas e as ferramentas de aprendizado de máquina e de aprendizado profundo. O segundo é **analítico**: usar essas etapas para entender um resultado negativo, o de uma rede que reconhece muito bem os glitches do Gravity Spy, mas não reconhece as imagens dos eventos reais.

Trata-se de uma revisão narrativa, apoiada na literatura de referência sobre detectores, processamento de sinais e aprendizado de máquina em ondas gravitacionais, combinada a um estudo de caso: o pipeline do Projeto 3 do PIC Astronomia, cujo código, configurações e relatórios são públicos. Cada conceito é apresentado primeiro em termos gerais e depois aplicado aos dados do projeto, com o GW150914 como exemplo recorrente.

A organização segue a ordem em que o projeto foi construído (Figura 1). A Seção 2 revisa os fundamentos: ondas gravitacionais, detectores e glitches. A Seção 3 trata dos dados e do condicionamento do sinal; a Seção 4, da validação física com o GW150914; e a Seção 5, das ferramentas de aprendizado de máquina. A Seção 6 apresenta o estudo de caso — a evolução do projeto e os resultados observados —, a Seção 7 discute as lições e as perspectivas, e a Seção 8 traz as considerações finais.

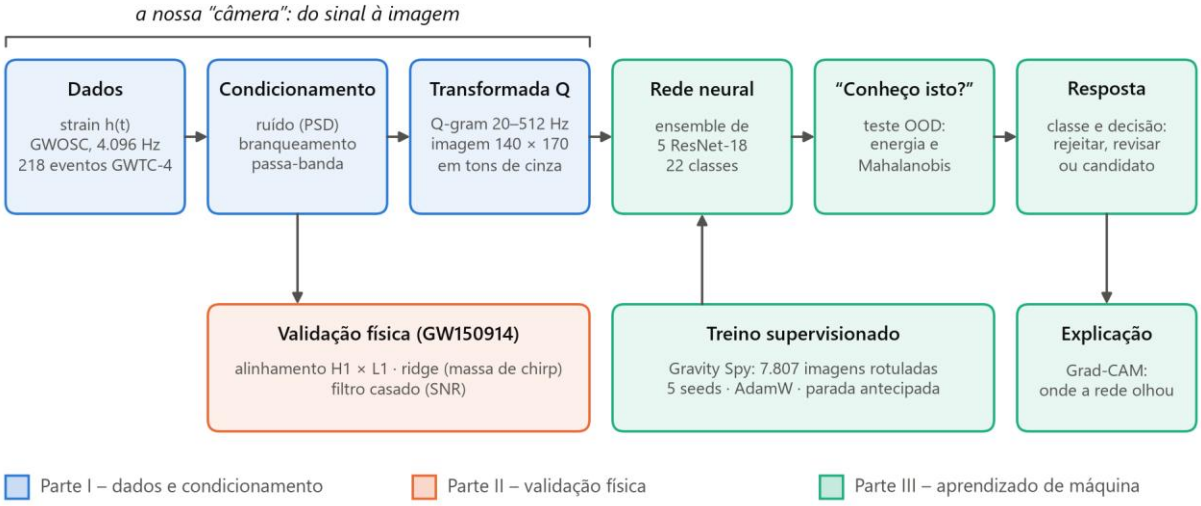


Figura 1 – Fluxo do estudo de caso. As três primeiras etapas formam a "câmera" do projeto, que transforma o sinal do detector numa imagem. A validação física usa o mesmo sinal condicionado; a rede neural é treinada com as imagens do Gravity Spy e aplicada às imagens da nossa câmera.

2. Fundamentos: ondas gravitacionais, detectores e glitches

2.1 Ondas gravitacionais e os detectores

Uma onda gravitacional é uma perturbação do espaço-tempo que se propaga à velocidade da luz [5]. Ao passar pela Terra, ela estica e encolhe as distâncias de forma alternada, em direções perpendiculares. Os interferômetros LIGO — Hanford (H1) e Livingston (L1), nos Estados Unidos — e Virgo (V1), na Itália, medem essa deformação comparando, com laser, o comprimento de dois braços de 4 km (3 km no Virgo). A grandeza medida é o *strain*:

$$h(t) = \frac{\Delta L(t)}{L} \tag{1}$$

em que L é o comprimento do braço e ΔL a variação causada pela onda. Para as fontes já detectadas, $h \sim 10^{-21}$: num braço de 4 km, a variação é centenas de vezes menor que o diâmetro de um próton. O GW150914 foi a fusão de dois buracos negros de cerca de 36 e 29 massas solares, a cerca de 410 Mpc [1]; ele é o exemplo usado ao longo deste artigo.

2.2 Glitches e o projeto Gravity Spy

Nessa sensibilidade, quase tudo perturba o detector: vibrações do solo, luz espalhada, falhas eletrônicas, a rede elétrica. Muitos desses distúrbios aparecem como transientes curtos — os glitches — que reduzem o tempo útil de observação e podem esconder ou imitar um sinal astrofísico [3]. Os glitches são encontrados por algoritmos de gatilho como o Omicron, que procura excessos de energia no plano tempo-frequência [6].

O projeto Gravity Spy [7, 8] combinou voluntários da plataforma Zooniverse e aprendizado de máquina para rotular esses gatilhos em 22 classes, segundo a forma que eles têm na imagem tempo-frequência; o conjunto foi depois estendido até a terceira corrida de observação, O3 [9]. A Tabela 1 descreve algumas classes. Essas imagens rotuladas são o material de treino da rede do estudo de caso.

Tabela 1 – Algumas das 22 classes do Gravity Spy

Classe	Aparência na imagem tempo × frequência e origem provável
Blip	pulso curto (~10 ms) e simétrico, em forma de gota; origem desconhecida
Koi_Fish	parecido com o Blip, porém mais intenso e mais largo em frequência
Tomte	pulso curto concentrado em baixa frequência
Whistle	assobio: trilha em V ou W em alta frequência, por batimento de rádio frequência
Scattered_Light	arcos em baixa frequência, de luz espalhada por superfícies em vibração
Low_Frequency_Lines	linhas horizontais em baixa frequência
Power_Line	linha da rede elétrica (60 Hz) e harmônicos
Violin_Mode	linhas perto de 500 Hz: ressonância dos fios que suspendem os espelhos
1400Ripples	ondulações perto de 1.400 Hz
Chirp	trilha que sobe em frequência, com formato parecido com o de uma coalescência
Extremely_Loud	excesso de energia muito alto, que satura a imagem
No_Glitch	só ruído de fundo, sem glitch
None_of_the_Above	glitch que não se encaixa em nenhuma outra classe

As demais classes são 1080Lines, Air_Compressor, Helix, Light_Modulation, Low_Frequency_Burst, Paired_Doves, Repeating_Blips, Scratchy e Wandering_Line.

2.3 A pergunta do estudo de caso

Redes treinadas nas imagens do Gravity Spy classificam bem os glitches. A pergunta do estudo de caso é de **validade externa**: uma rede treinada nessas imagens consegue dizer algo útil sobre as imagens dos eventos reais do GWTC-4? Se conseguisse, seria uma ferramenta automática para examinar a vizinhança de cada detecção; se não, é preciso entender por quê, para que ninguém confie nas respostas dela sem verificação. Vale deixar claro desde já: a rede classifica a **aparência** de glitches. Ela não foi treinada para detectar ondas gravitacionais, e a classe Chirp do Gravity Spy descreve um formato, não uma origem astrofísica.

> **Em poucas palavras.** Os detectores medem uma deformação minúscula do espaço; glitches são ruídos que imitam sinais. O Gravity Spy rotulou milhares deles por forma. O estudo de caso pergunta se uma rede treinada nessas imagens funciona nas imagens dos eventos reais.

3. Do sinal à imagem: dados e condicionamento

3.1 De onde vêm os dados

Os dados abertos dos detectores são publicados pelo GWOSC (*Gravitational Wave Open Science Center*) [10]. No estudo de caso, para cada evento do catálogo baixamos 32 s de strain em torno do instante da fusão, amostrados a 4.096 Hz (131.072 amostras por detector), em arquivos HDF5. O GWTC-4 lista 218 eventos; com três detectores possíveis, são 654 combinações evento–detector. Um controle de qualidade decide quais canais podem ser usados (Tabela 2):

- nenhum valor não finito (NaN ou infinito) no meio do trecho; nas bordas, um recorte é permitido se sobrarem pelo menos 8 s válidos antes e depois do evento e 20 s contínuos;
- no máximo 1% de amostras nulas e nenhuma sequência de zeros maior que 0,25 s;
- o sinal branqueado (Seção 3.3) precisa ter desvio-padrão entre 0,90 e 1,10.

Tabela 2 – O catálogo GWTC-4 depois do controle de qualidade

Item	Quantidade
Eventos no catálogo	218
Combinações evento–detector	654
Canais aprovados	475
Sem dados públicos no GWOSC	153
Com valores não finitos no trecho	21
Reprovados no branqueamento	5
Eventos com dois ou mais detectores / com um detector	204 / 14

3.2 O ruído do detector: densidade espectral de potência

O ruído não é igual em todas as frequências. A forma padrão de descrevê-lo é a densidade espectral de potência (PSD, *power spectral density*) [3], estimada pelo método de Welch [11]: o trecho é dividido em segmentos (no projeto, de 4 s, com 50% de sobreposição), cada segmento é multiplicado por uma janela de Hann, e os espectros são combinados:

$$S_n(f) = \frac{2}{f_s \sum_n w_n^2} \cdot \frac{1}{K} \sum_{k=1}^{K} |\tilde{x}_k(f)|^2 \qquad (2)$$

Aqui $\tilde{x}_k(f)$ é a transformada de Fourier do k-ésimo segmento, já multiplicado pela janela w; f_s é a taxa de amostragem; e o fator 2 soma as frequências negativas às positivas (espectro unilateral). No projeto, a média sobre os K segmentos é trocada pela **mediana**, mais robusta quando há um glitch dentro do trecho. A raiz, $A(f) = \sqrt{S_n(f)}$, é a densidade espectral de amplitude (ASD), mostrada na Figura 2 para o GW150914.

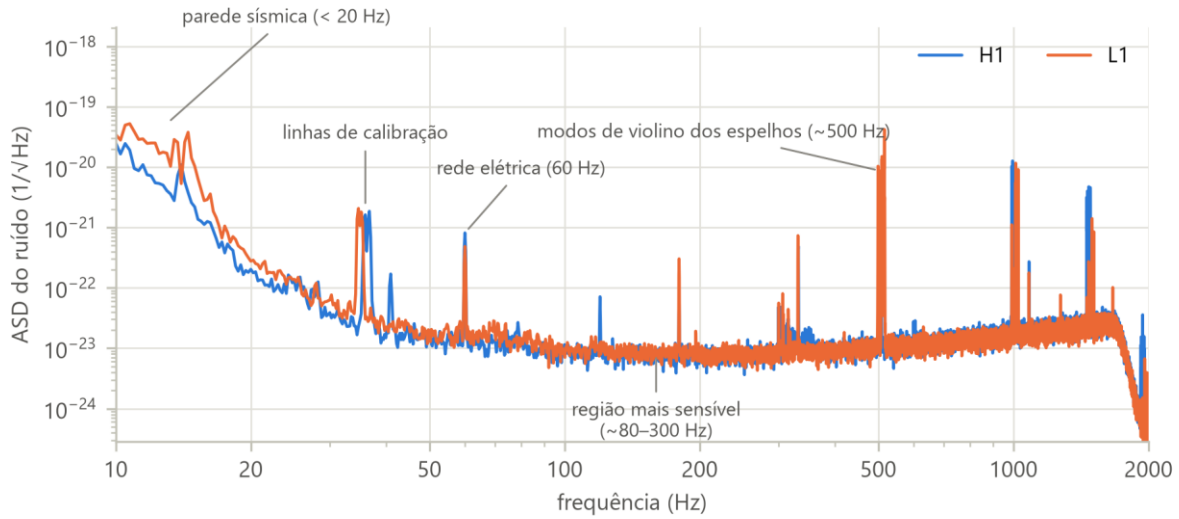


Figura 2 – Ruído dos detectores H1 e L1 no trecho do GW150914 (ASD). A parede sísmica domina abaixo de ~20 Hz; os picos estreitos são linhas instrumentais (calibração, rede elétrica de 60 Hz, modos de violino dos fios de suspensão perto de 500 Hz e seus harmônicos). A região mais sensível fica entre ~80 e 300 Hz. A queda acima de ~1.700 Hz vem do filtro antialiasing da amostragem em 4.096 Hz.

3.3 Branqueamento

O branqueamento divide o sinal, no domínio da frequência, pelo tamanho típico do ruído em cada frequência [3]:

$$\tilde{x}_w(f) = \frac{\tilde{x}(f)}{\sqrt{S_n(f)/2}}, \qquad 10\ \text{Hz} \le f \le 2000\ \text{Hz} \tag{3}$$

Antes da transformada, o trecho é multiplicado por uma janela de Tukey (α = 0,125), que leva as bordas suavemente a zero; fora da banda de 10–2.000 Hz, o espectro é zerado; depois, a transformada inversa devolve o sinal ao domínio do tempo. O efeito é deixar o ruído "branco": todas as frequências passam a contribuir com a mesma potência, e o que sobressai é excesso real de energia — um glitch ou uma onda. Como verificação, o desvio-padrão do sinal branqueado (sem as bordas) deve ficar perto de 1; no GW150914 ele foi 0,989 (H1) e 0,987 (L1).

> **Em poucas palavras.** Branquear é como equalizar uma música com grave demais: abaixamos as frequências em que o ruído é alto para que um detalhe fraco em outra frequência apareça.

3.4 Filtros passa-banda e rejeita-faixa

Em seguida, um filtro passa-banda Butterworth de ordem n = 4 mantém só a faixa entre f_1 = 20 Hz e f_2 = 1.024 Hz. A resposta de amplitude desse filtro é

$$|H(f)|^2 = \frac{1}{1 + \left(\dfrac{f^2 - f_0^2}{f\,\Delta f}\right)^{2n}}, \qquad f_0 = \sqrt{f_1 f_2}, \quad \Delta f = f_2 - f_1 \tag{4}$$

O filtro é aplicado duas vezes, para a frente e para trás (função `sosfiltfilt` do SciPy), o que dobra a atenuação e cancela o atraso de fase: o sinal não é deslocado no tempo. Para remover linhas estreitas (60 Hz e harmônicos, por exemplo), usam-se filtros rejeita-faixa (*notch*) com fator de qualidade Q = 30:

$$H(s) = \frac{s^2 + \omega_0^2}{s^2 + (\omega_0/Q)\,s + \omega_0^2} \tag{5}$$

em que $\omega_0 = 2\pi f_0$ é a frequência removida. (Esse Q é o do filtro, não o da transformada da próxima seção.)

3.5 A transformada Q e o Q-gram

Para a rede neural, o sinal precisa virar imagem. Um espectrograma comum (transformada de Fourier de tempo curto) usa uma janela de duração fixa. A transformada Q [12] usa uma janela que encolhe à medida que a frequência sobe:

$$X(\tau, f, Q) = \int_{-\infty}^{\infty} x(t)\, w(t - \tau;\, f, Q)\, e^{-2\pi i f t}\, dt \tag{6}$$

$x(t)$ é o sinal branqueado; τ é o instante analisado; f é a frequência; w é uma janela centrada em τ cuja largura é proporcional a Q/f; e o termo exponencial mede quanto o sinal oscila na frequência f dentro da janela. O fator Q é, aproximadamente, o número de ciclos da onda dentro da janela, e fica constante:

$$\Delta t \propto \frac{Q}{f}, \qquad \Delta f \propto \frac{f}{Q} \tag{7}$$

Em frequências baixas, a janela é longa (boa resolução em frequência); em frequências altas, é curta (boa resolução no tempo). Isso combina com glitches curtos e com chirps, cuja frequência sobe rapidamente. Na prática, a biblioteca gwpy [13] calcula a integral como uma soma discreta no domínio

da frequência, com uma janela "bisquare" de suporte finito no lugar da gaussiana teórica. A imagem não mostra $|X|^2$ diretamente, mas a **energia normalizada**:

$$Z(\tau, f) = \frac{|X(\tau, f, Q)|^2}{\text{mediana}_\tau |X(\tau, f, Q)|^2} \tag{8}$$

$Z \approx 1$ é ruído típico; valores altos indicam excesso de energia. O cálculo é repetido para Q entre 4 e 64, e fica o Q que concentra mais energia:

$$Q^* = \arg\max_{Q \in [4, 64]} \max_{\tau, f} Z(\tau, f; Q) \tag{9}$$

No projeto, o Q-gram cobre 20–512 Hz em 500 frequências espaçadas logaritmicamente, numa janela de 4 s com 1.000 instantes; a rede usa o segundo central. A Figura 3 mostra o caminho completo com o GW150914 no L1.

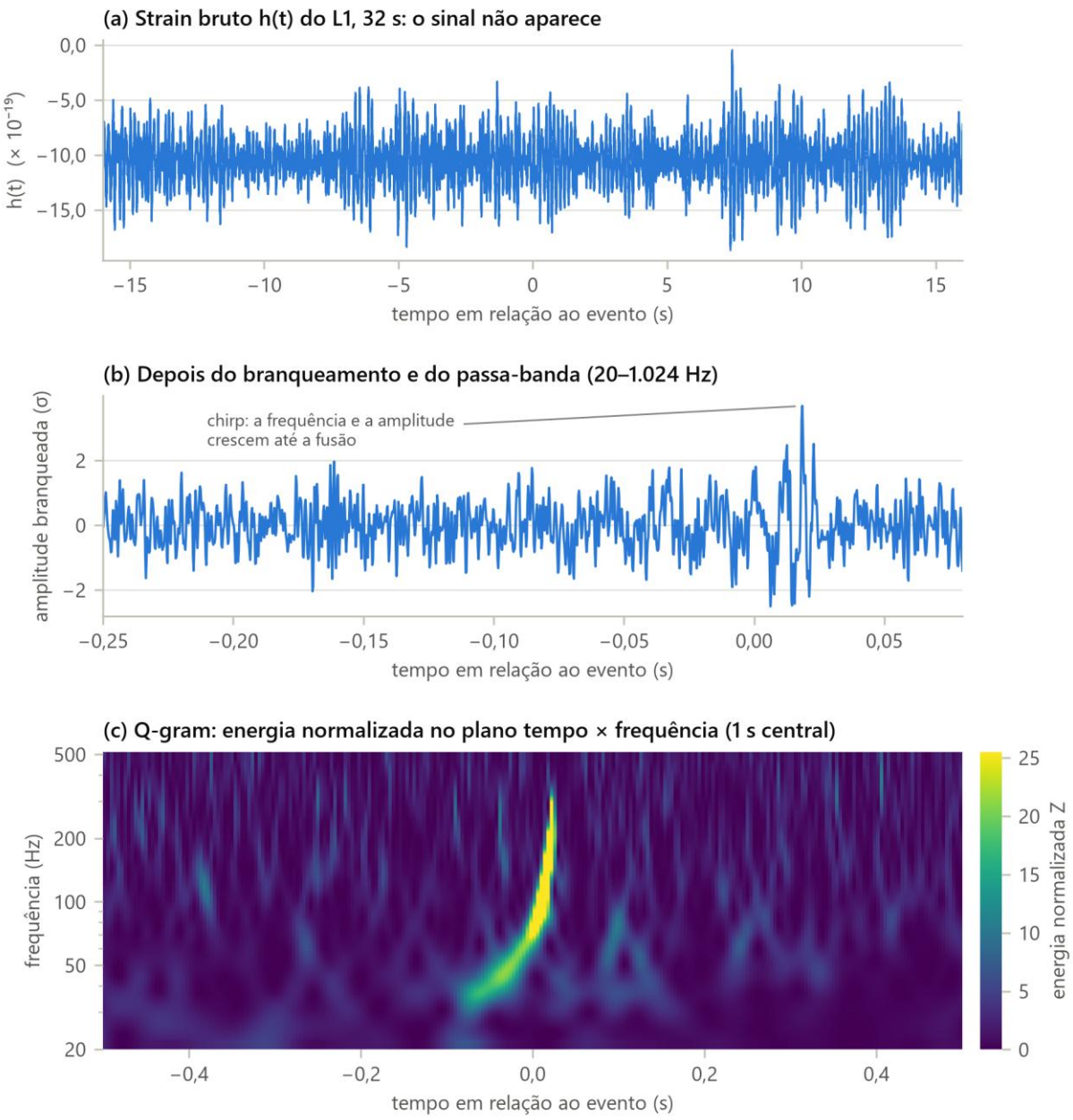


Figura 3 – GW150914 no detector L1, do strain bruto ao Q-gram. (a) No dado bruto, dominado pelo ruído de baixa frequência, o sinal não aparece. (b) Depois do branqueamento e do passa-banda, surge a oscilação que cresce até a fusão. (c) No Q-gram, a energia desenha a trilha característica do chirp, que sobe de ~35 Hz para ~300 Hz em cerca de 0,1 s.

3.6 Da matriz à imagem: renderização

O Q-gram é uma matriz de números. Para a rede, ele vira uma imagem de 140 × 170 pixels em tons de cinza, com as frequências altas no topo, como nas imagens do Gravity Spy. O projeto testou duas formas de converter energia em tons de cinza. A primeira, por **percentis**, era a forma padrão da linha de código 2.0:

$$E = \ln\left(1 + \max\left(\frac{Z}{\text{mediana}_t Z} - 1;\ 0\right)\right) \tag{10}$$

$$I = 255 \cdot \text{clip}\left(\frac{E - P_1}{P_{99,5} - P_1};\ 0;\ 1\right) \tag{11}$$

P_1 e $P_{99,5}$ são os percentis 1 e 99,5 da própria imagem: o contraste é esticado imagem a imagem; a função clip limita o valor ao intervalo indicado (aqui, os argumentos são separados por ponto e vírgula, porque a vírgula é a decimal). A segunda forma, a **escala Gravity Spy**, repete a cadeia dos PNGs de treino:

$$I = \text{cinza}\left(\text{viridis}\left(\text{clip}(Z/25{,}5;\ 0;\ 1)\right)\right) \tag{12}$$

A energia de 0 a 25,5 é pintada com o mapa de cores viridis e depois convertida para cinza. Com ela, o fundo fica cinza-escuro, como nas imagens que a rede conhece; com os percentis, o fundo fica preto e o ruído é realçado (Figura 4).

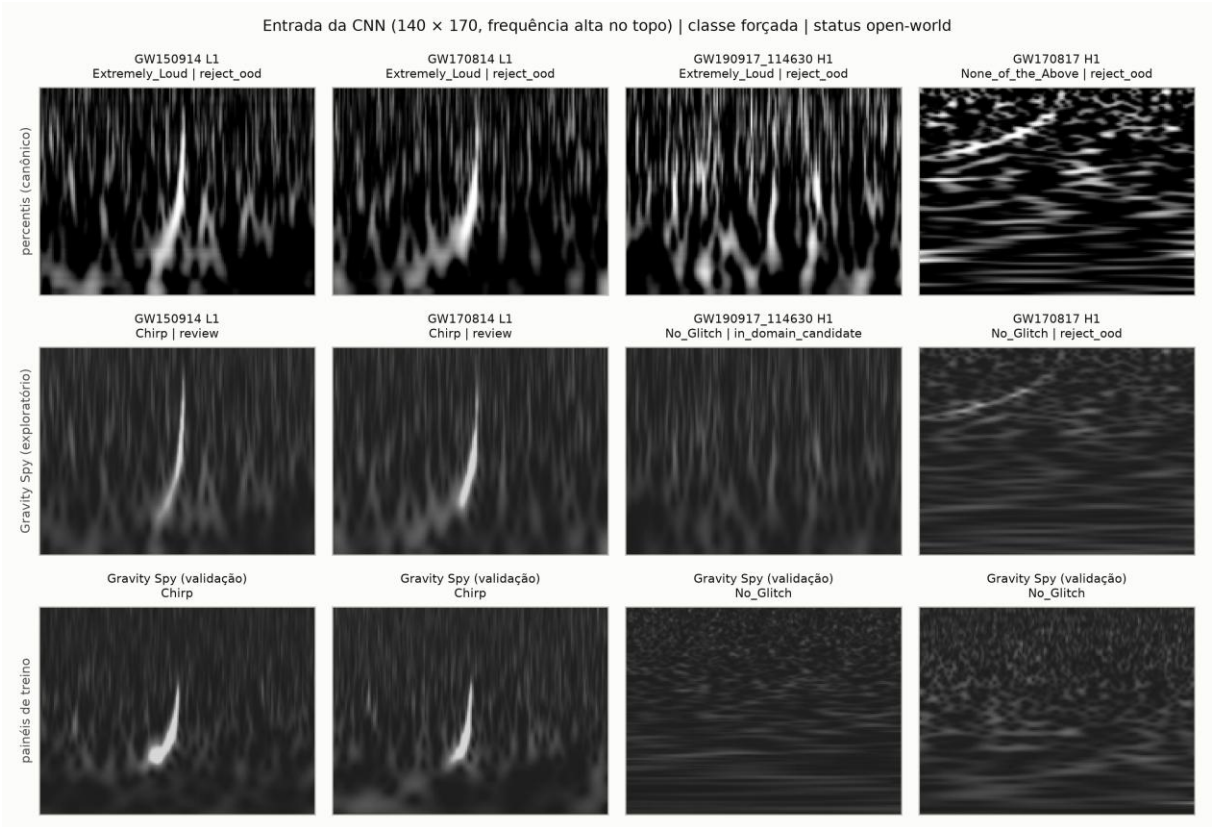


Figura 4 – O que a rede vê. Linha de cima: eventos com a renderização por percentis. Linha do meio: os mesmos eventos na escala Gravity Spy. Linha de baixo: imagens de validação do Gravity Spy, que a rede conhece. Acima de cada painel, a classe forçada e a decisão do teste de "fora da distribuição" (Seção 5.9).

> **Em poucas palavras.** A "câmera" do projeto tem três passos: medir o ruído, branquear e filtrar o sinal, e transformá-lo numa imagem tempo × frequência. Cada escolha — janela, faixa de frequência, escala de cinza — muda a aparência da imagem, e uma rede neural é sensível a essa aparência.

4. Validação física: o GW150914 como referência

Antes de usar um pipeline num catálogo, convém verificar se ele recupera a física de um evento conhecido. O GW150914 é o melhor caso: é forte, foi visto pelos dois LIGO e tem parâmetros bem medidos. O projeto fez três verificações: o alinhamento entre detectores, a massa de chirp pela trilha do sinal (*ridge*) e o filtro casado.

4.1 A física do chirp

Dois objetos compactos em órbita perdem energia emitindo ondas gravitacionais, se aproximam e giram cada vez mais rápido. A frequência da onda (o dobro da frequência orbital) e a amplitude sobem até a fusão: é o *chirp*. Na aproximação newtoniana, a evolução depende de uma única combinação das massas, a **massa de chirp** [5]:

$$\mathcal{M} = \frac{(m_1 m_2)^{3/5}}{(m_1 + m_2)^{1/5}} \tag{13}$$

$$\frac{df}{dt} = \frac{96}{5}\,\pi^{8/3}\left(\frac{G\mathcal{M}}{c^3}\right)^{5/3} f^{11/3} \tag{14}$$

Integrando, $f^{-8/3}$ varia linearmente com o tempo até o instante t_c da coalescência:

$$f^{-8/3}(t) = \frac{256}{5}\,\pi^{8/3}\left(\frac{G\mathcal{M}}{c^3}\right)^{5/3}(t_c - t) \tag{15}$$

A fase de espiralamento termina perto da órbita circular estável mais interna (ISCO), cuja frequência de onda é aproximadamente

$$f_{\mathrm{ISCO}} \approx \frac{c^3}{6^{3/2}\,\pi\, GM} \tag{16}$$

com $M = m_1 + m_2$. Para o GW150914, com m_1 = 38,1 e m_2 = 33,0 massas solares no referencial do detector (as massas observadas são multiplicadas por $1 + z$, em que z é o desvio para o vermelho), $f_{\mathrm{ISCO}} \approx$ 62 Hz, e a massa de chirp de referência do catálogo é 30,7 massas solares.

4.2 Alinhamento entre detectores

Uma onda viaja à velocidade da luz; entre Hanford e Livingston, a cerca de 3.000 km um do outro, o atraso máximo é de ~10 ms. Como os braços dos dois detectores têm orientações diferentes, o sinal num deles aparece invertido em relação ao outro [1]. Para medir o atraso, calcula-se a correlação cruzada normalizada entre os dois sinais branqueados e filtrados em 35–350 Hz, deslocando um deles amostra a amostra:

$$r(\ell) = \frac{\sum_n (a_{n+\ell} - \overline{a})(b_n - \overline{b})}{N\,\sigma_a\,\sigma_b}, \qquad \ell^* = \underset{|\ell| \le \ell_{\max}}{\arg\max} |r(\ell)| \tag{17}$$

O sinal de $r(\ell^*)$ diz se é preciso inverter um dos sinais. No GW150914 (Figura 5), o melhor alinhamento é o L1 deslocado 7,3 ms e invertido, com correlação de 0,77 na janela de −0,12 a +0,04 s, compatível com os 6,9 ms publicados pela colaboração [1].

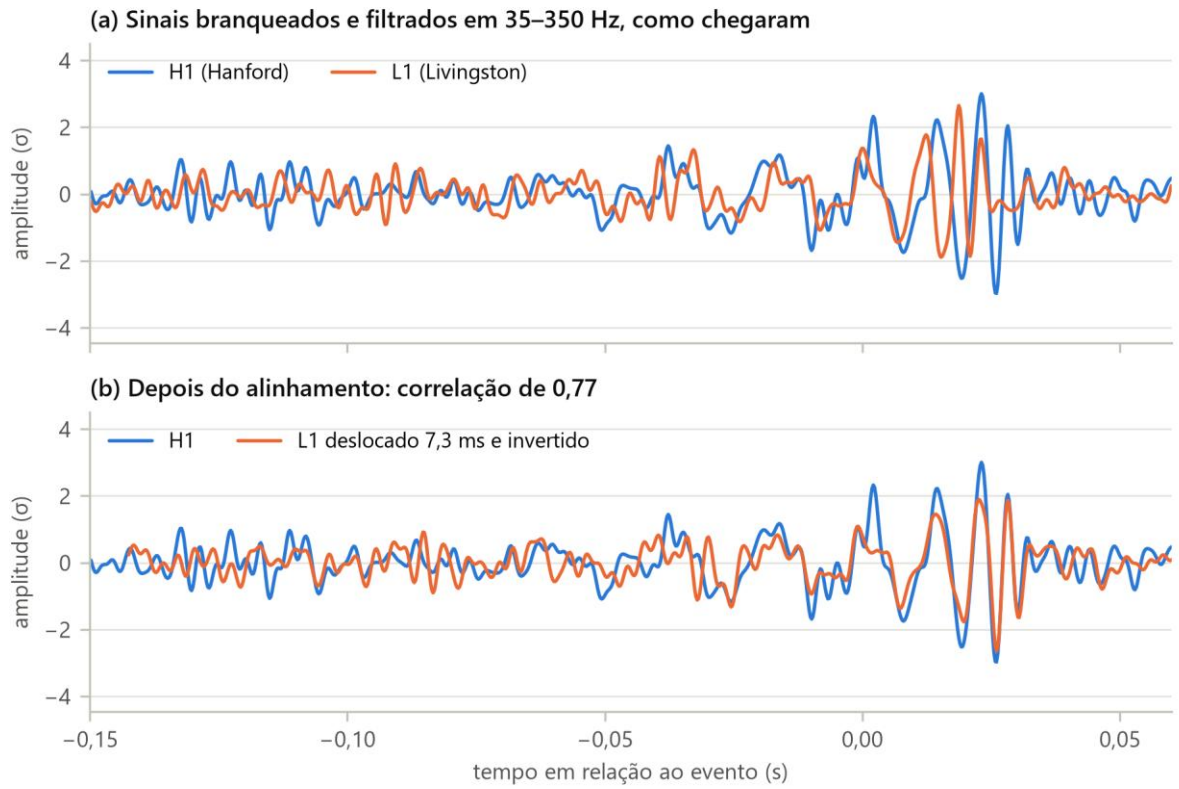


Figura 5 – Alinhamento entre detectores no GW150914. (a) Sinais branqueados e filtrados em 35–350 Hz, como chegaram. (b) O L1 deslocado 7,3 ms e invertido se sobrepõe ao H1 (correlação de 0,77).

Uma verificação mais exigente, feita na versão v8 do pipeline — comparar a forma de onda filtrada, sem branqueamento, com a da Figura 1 do artigo de descoberta —, não passou: a correlação foi de 0,44–0,46, para um critério de 0,95. Esse teste não foi retomado nas versões seguintes.

4.3 Massa de chirp pela trilha do sinal (ridge)

O *ridge* é a trilha de máxima energia no Q-gram. O algoritmo do projeto (i) define uma janela a partir da massa do catálogo e limita a frequência pela ISCO; (ii) usa Q = 5 fixo; (iii) extrai um caminho quase monotônico por programação dinâmica; (iv) ajusta a reta da Equação (15) com o estimador robusto de Theil–Sen [14]; e (v) converte a inclinação s em massa de chirp:

$$\mathcal{M} = \frac{c^3}{G}\left(\frac{5\,|s|}{256}\right)^{3/5} \pi^{-8/5} \tag{18}$$

A Figura 6 mostra o GW150914. O ridge do H1 pega, além do chirp, trechos de ruído antes dele (os segmentos horizontais em ~30 e ~50 Hz), e a inclinação ajustada dá 21,9 massas solares; no L1, 10,7 — contra 30,7 do catálogo. No catálogo inteiro, 201 dos 475 canais tiveram ajuste aceito, com erro relativo mediano de 34,7%. Na validação da versão v8, com outra janela, o GW150914 deu 35,5 (H1) e 41,6 (L1) massas solares, erros de 16% e 36%, acima do limite de 15% fixado para o teste.

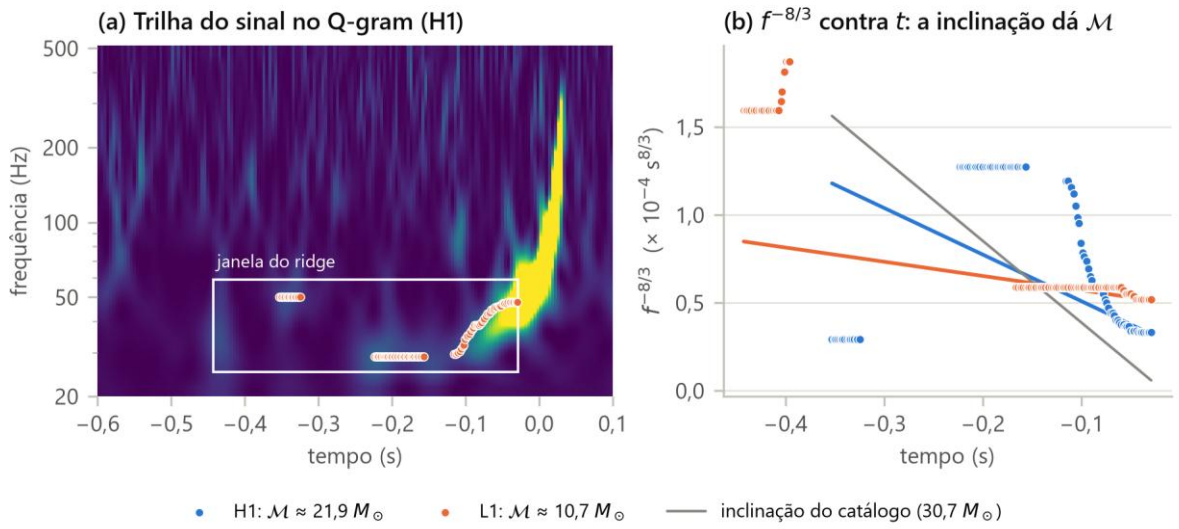


Figura 6 – Massa de chirp pelo ridge no GW150914. (a) Pontos do ridge (laranja) sobre o Q-gram do H1; o retângulo é a janela de busca. (b) $f^{-8/3}$ contra o tempo, com as retas de Theil–Sen do H1 e do L1 e a inclinação esperada pela massa do catálogo (cinza).

4.4 Filtro casado

O filtro casado compara o dado com um molde teórico da onda (*template*) e é o método ótimo para detectar um sinal conhecido em ruído gaussiano estacionário [15]. A relação sinal-ruído (SNR) em função do tempo é

$$\rho(t) = \frac{\left| 4 \int_{f_{\text{low}}}^{f_{\text{high}}} \frac{\tilde{d}(f)\,\tilde{h}^{*}(f)}{S_n(f)}\, e^{2\pi i f t}\, df \right|}{\sqrt{4 \int_{f_{\text{low}}}^{f_{\text{high}}} \frac{|\tilde{h}(f)|^2}{S_n(f)}\, df}} \tag{19}$$

$\tilde{d}$ é o dado no domínio da frequência, $\tilde{h}$ é o template — no projeto, a aproximação pós-newtoniana TaylorF2 [16], só a fase de espiralamento — e o denominador normaliza o template. Para combinar os detectores, $\rho_{\text{rede}} = \sqrt{\rho_{\text{H1}}^2 + \rho_{\text{L1}}^2}$. A Tabela 3 mostra o teste do GW150914 com cinco templates.

Tabela 3 – Filtro casado no GW150914 (versão v8; só a fase de espiralamento)

Template	SNR H1	SNR L1	SNR de rede
BBH 36 + 29 (o do evento)	**6,5**	**5,4**	**8,5**
BBH assimétrico 30 + 10	6,1	4,2	7,5
BBH leve 10 + 10	4,8	4,5	6,6
NSBH 10 + 1,4	3,5	4,3	5,5
BNS 1,4 + 1,4	3,6	3,4	5,0

BBH: binária de buracos negros; NSBH: buraco negro e estrela de nêutrons; BNS: binária de estrelas de nêutrons. Massas em massas solares.

O template certo venceu, mas com vantagem de só 1,13 vez sobre o segundo, e o critério de margem não passou. A SNR publicada, com templates completos (espiralamento, fusão e relaxação), é cerca de 24 [1]: usar só o espiralamento, até $f_{\text{ISCO}} \approx 62$–$69$ Hz, descarta a parte mais forte do sinal. No catálogo (linha 2.0), 164 eventos foram analisados e nenhum passou no critério completo do diagnóstico.

> **Em poucas palavras.** O pipeline recupera a física do GW150914 de forma qualitativa — o atraso e a inversão entre detectores, a trilha do chirp, o template certo em primeiro lugar —, mas as medidas quantitativas (massa de chirp, SNR) ficam longe dos valores publicados. Essas ferramentas servem como verificação do pipeline, não como medição.

5. Aprendizado de máquina e aprendizado profundo

5.1 Panorama: aprendizado de máquina em ondas gravitacionais

O uso de aprendizado de máquina em ondas gravitacionais cresceu muito a partir de 2017 [4]. Três frentes se destacam. A primeira é a **classificação de glitches**: o Gravity Spy combinou classificação por voluntários e redes convolucionais [7, 8], e outros trabalhos mostraram que redes aplicadas a imagens tempo-frequência classificam transientes de ruído com alta acurácia [17]. A segunda é a **busca de sinais**: redes treinadas com sinais simulados em ruído chegam a sensibilidades comparáveis às do filtro casado para binárias de buracos negros [18, 19]; estratégias de treino foram estudadas de forma sistemática [20], e um desafio comparou diversos algoritmos em condições padronizadas [21]. A terceira é a **separação entre sinal e glitch** em candidatos reais, como no GSpyNetTree [22].

O estudo de caso se situa na primeira frente, mas toca num problema que atravessa as três: a **mudança de domínio**, isto é, a diferença entre os dados usados no treino e os dados em que a rede é usada. As subseções a seguir revisam as ferramentas, na ordem em que aparecem no pipeline.

5.2 Conceitos básicos

Aprendizado de máquina (*machine learning*, ML) é ajustar os parâmetros de um modelo a partir de exemplos, em vez de programar as regras à mão. No aprendizado **supervisionado**, cada exemplo vem com a resposta certa (o rótulo), e o modelo aprende a mapear a entrada no rótulo. Aprendizado profundo (*deep learning*, DL) é o caso em que o modelo é uma rede neural com muitas camadas, que aprende sozinha quais características da entrada importam [23].

Os dados são divididos em três conjuntos que não se misturam: **treino** (ajusta os pesos), **validação** (escolhe a melhor época, calibra a confiança e os limiares) e **teste** (mede o desempenho final, uma única vez). No estudo de caso, depois de remover 159 painéis vazios repetidos do Gravity Spy, ficaram 7.807 imagens: 5.472 de treino, 1.177 de validação e 1.158 de teste.

5.3 Uma linha de base de ML clássico

Antes da rede neural, vale medir o que um modelo simples consegue. O projeto treinou uma regressão logística multinomial [24] sobre os pixels da imagem reduzida a 32 × 32:

$$p(c \mid \mathbf{x}) = \frac{e^{\mathbf{w}_c^{\top}\mathbf{x}+b_c}}{\sum_{j=1}^{C} e^{\mathbf{w}_j^{\top}\mathbf{x}+b_j}} \tag{20}$$

com regularização L2 (C = 0,1) e pesos balanceados por classe. Ela chega a macro-F1 de 0,797: as classes do Gravity Spy já são bastante separáveis por brilho e forma grosseira. A rede neural precisa superar essa referência para justificar sua complexidade.

5.4 Os blocos de uma rede convolucional

Convolução. Um filtro K (por exemplo, 3 × 3) desliza pela imagem e responde a padrões locais — bordas, manchas, trilhas [23]:

$$(K * x)_{i,j} = \sum_{u} \sum_{v} K_{u,v}\, x_{i+u,\, j+v} + b \tag{21}$$

Cada camada tem dezenas a centenas de filtros, e cada filtro produz um **mapa de ativação**. Os valores de K e b não são escolhidos à mão: são aprendidos no treino.

Ativação ReLU. Depois de cada convolução, os valores negativos são zerados, o que dá à rede a capacidade de representar relações não lineares:

$$\mathrm{ReLU}(z) = \max(0,\, z) \tag{22}$$

Normalização em lote (*batch normalization*) [25]. Padroniza cada canal com a média μ_B e a variância σ_B^2 do lote e reescala com dois parâmetros aprendidos, γ e β; isso estabiliza e acelera o treino:

$$\hat{z} = \gamma\, \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta \tag{23}$$

Bloco residual [26]. Um atalho soma a entrada à saída do bloco; assim, o bloco só precisa aprender uma correção $F(\mathbf{x})$, e redes profundas podem ser treinadas sem que o gradiente desapareça:

$$\mathbf{y} = \mathrm{ReLU}(F(\mathbf{x}) + \mathbf{x}) \tag{24}$$

Média espacial global (GAP). No fim, cada mapa de ativação A^k vira um único número, a sua média:

$$g_k = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{i,j}^{k} \tag{25}$$

Softmax. A última camada produz um número por classe, o *logit* z_c; a função softmax transforma os logits em probabilidades que somam 1:

$$p_c = \frac{e^{z_c/T}}{\sum_{j=1}^{C} e^{z_j/T}} \tag{26}$$

T é a temperatura: vale 1 no treino e é calibrada depois (Seção 5.7).

5.5 A arquitetura: ResNet-18 adaptada

O estudo de caso usa a ResNet-18 [26] da biblioteca torchvision [27], pré-treinada no ImageNet [28] — um caso de **aprendizado por transferência**: a rede começa com filtros que já reconhecem bordas e texturas. Foram feitas três adaptações: (i) a primeira convolução passa a ter um canal de entrada (imagens em cinza), com pesos iniciais iguais à soma dos pesos RGB, o que preserva a resposta a uma imagem cinza; (ii) o *max-pooling* inicial é removido, para não perder resolução numa imagem pequena; (iii) a última camada passa a ter 22 saídas, uma por classe. A rede tem 11.181.526 parâmetros (Tabela 4 e Figura 7).

Tabela 4 – Camadas da ResNet-18 adaptada e o tamanho da saída de cada uma

Camada	Operação	Saída (canais × altura × largura)
Entrada	imagem em cinza, normalizada para [−1, 1]	1 × 140 × 170
conv1	convolução 7 × 7, 64 filtros, passo 2; BN; ReLU	64 × 70 × 85
layer1	2 blocos residuais (convoluções 3 × 3, 64 filtros)	64 × 70 × 85
layer2	2 blocos residuais (3 × 3, 128 filtros), passo 2	128 × 35 × 43
layer3	2 blocos residuais (3 × 3, 256 filtros), passo 2	256 × 18 × 22
layer4	2 blocos residuais (3 × 3, 512 filtros), passo 2	512 × 9 × 11
GAP	média espacial de cada mapa de ativação	512
FC	camada totalmente conectada 512 → 22, seguida de softmax	22

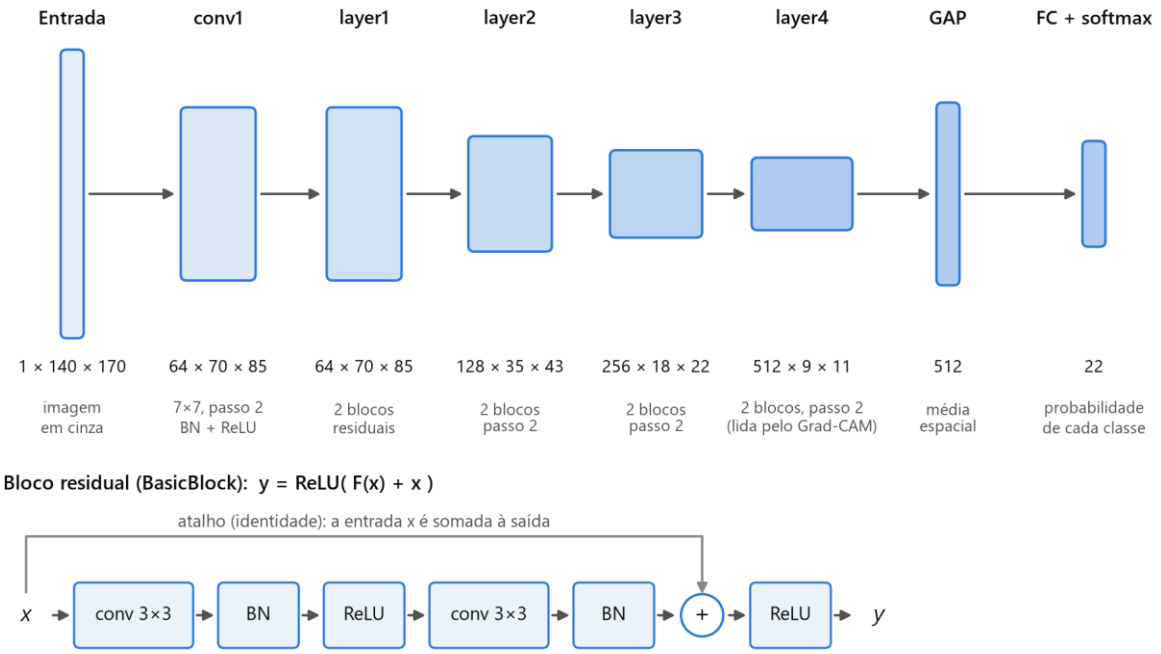


Figura 7 – ResNet-18 adaptada. A altura dos blocos é proporcional à altura do mapa de ativação: cada etapa reduz a resolução pela metade e dobra o número de filtros. Embaixo, o bloco residual básico, com o atalho que soma a entrada à saída.

Nas primeiras camadas, os filtros veem detalhes pequenos; nas últimas, cada posição do mapa de 9 × 11 "enxerga" uma região grande da imagem e responde a formas inteiras — uma trilha, um arco, uma linha. É por isso que o Grad-CAM (Seção 5.10) lê a layer4.

5.6 O treino

A função de perda é a **entropia cruzada ponderada**. O peso w_c compensa as classes raras (N_c é o número de exemplos de treino da classe c; os pesos são normalizados para média 1):

$$\mathcal{L} = -\frac{\sum_{i=1}^{N} w_{y_i} \ln p_{y_i}(\mathbf{x}_i)}{\sum_{i=1}^{N} w_{y_i}}, \qquad w_c \propto \frac{N}{N_c} \tag{27}$$

Os pesos θ da rede são atualizados pelo otimizador AdamW [29, 30], que combina uma média móvel do gradiente $\mathbf{g}_t$ (o "momento") com uma escala por parâmetro:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2 \tag{28}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\, \theta_{t-1} \right) \tag{29}$$

$\hat{\mathbf{m}}_t$ e $\hat{\mathbf{v}}_t$ são as médias corrigidas, $\mathbf{m}_t/(1-\beta_1^t)$ e $\mathbf{v}_t/(1-\beta_2^t)$; η é a taxa de aprendizado; e λ é o decaimento dos pesos, aplicado separado do gradiente — a diferença entre o AdamW e o Adam original. A Tabela 5 resume as escolhas do treino.

Tabela 5 – Escolhas do treino no estudo de caso (configs/gravity_spy.toml)

Item	Valor e motivo
Fases	épocas 1–5: só conv1, bn1 e a camada final são treinadas (base pré-treinada congelada); depois, a rede inteira é ajustada (*fine-tuning*)
Taxa de aprendizado	10^{-3} na camada final; 10^{-4} na base durante o *fine-tuning*
Redução da taxa	pela metade quando o macro-F1 de validação não melhora por 2 épocas
Parada antecipada	até 30 épocas, parando após 7 sem melhora; fica a melhor época
Decaimento dos pesos	$\lambda = 10^{-4}$; $\beta_1 = 0{,}9$ e $\beta_2 = 0{,}999$ (padrão do PyTorch)
Aumento de dados	intensidade multiplicada por um fator aleatório entre 0,9 e 1,1
Lote e precisão	32 imagens por lote; precisão mista (AMP) na GPU RTX 3060
Reprodutibilidade	treino determinístico: mesma seed, mesmos pesos, bit a bit
Ensemble	5 seeds independentes: 11, 23, 42, 71 e 101

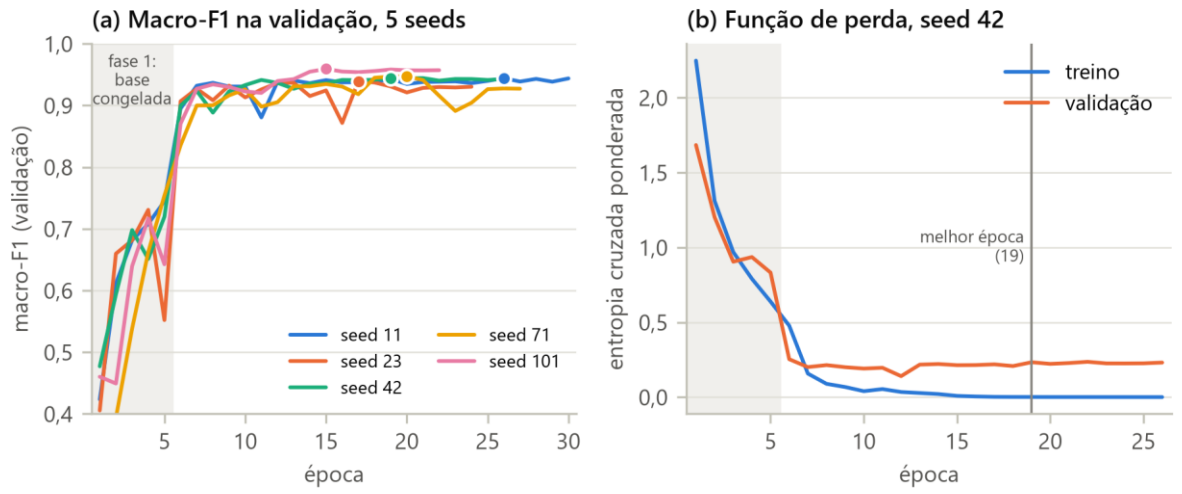


Figura 8 – Treino das 5 seeds. (a) Macro-F1 na validação por época; o ponto marca a melhor época de cada seed (entre 15 e 26). A faixa cinza é a fase com a base congelada. (b) Perda de treino e de validação da seed 42: a partir da época ~7, a perda de treino continua caindo e a de validação estabiliza — mais épocas só fariam a rede decorar o treino; por isso a parada antecipada.

5.7 Métricas e calibração

Para cada classe c, contam-se os verdadeiros positivos (VP), falsos positivos (FP) e falsos negativos (FN), e calculam-se precisão, revocação e F1:

$$P_c = \frac{\mathrm{VP}_c}{\mathrm{VP}_c + \mathrm{FP}_c}, \qquad R_c = \frac{\mathrm{VP}_c}{\mathrm{VP}_c + \mathrm{FN}_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c} \tag{30}$$

$$\text{macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c\,, \qquad \text{acurácia balanceada} = \frac{1}{C}\sum_{c=1}^{C} R_c \tag{31}$$

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{32}$$

O **macro-F1** dá o mesmo peso a todas as classes — uma classe rara mal classificada pesa tanto quanto uma comum —, por isso é a métrica principal. A acurácia comum (fração de acertos) é relatada ao lado. O kappa de Cohen [31] compara o acerto observado p_o com o acerto esperado por acaso p_e.

Uma rede pode acertar e ainda assim ser confiante demais. A **calibração** verifica se uma confiança de 80% corresponde a 80% de acertos. As previsões são agrupadas em B = 15 faixas de confiança, e o erro esperado de calibração (ECE) é [32]

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} |\text{acc}(B_b) - \text{conf}(B_b)| \tag{33}$$

Para corrigir, usa-se o **escalonamento por temperatura** [32]: um único número T, ajustado na validação para minimizar a log-verossimilhança negativa, divide os logits na Equação (26). No estudo de caso, T ficou entre 1,25 e 1,55 nas 5 seeds — a rede era confiante demais —, e o ECE da seed 42 caiu de 0,019 para 0,011. Os intervalos de confiança de 95% vêm de *bootstrap* [33] com 2.000 reamostragens; nos testes de fechamento, reamostram-se arquivos inteiros, porque glitches do mesmo arquivo não são independentes.

5.8 Ensemble e incerteza

Treinar várias redes com seeds diferentes e combinar as respostas — um *ensemble* — melhora a robustez e dá uma medida de incerteza [34]. No estudo de caso, M = 5 redes votam, e a classe final é a de maior probabilidade média:

$$\overline{p}_c = \frac{1}{M}\sum_{m=1}^{M} p_c^{(m)} \tag{34}$$

$$H(\mathbf{p}) = -\frac{1}{\ln C}\sum_{c=1}^{C} p_c \ln p_c\,, \qquad \text{IM} = H(\overline{\mathbf{p}}) - \frac{1}{M}\sum_{m=1}^{M} H\left(\mathbf{p}^{(m)}\right) \tag{35}$$

A entropia H, normalizada entre 0 e 1, mede a dúvida da resposta. A **informação mútua** IM mede o desacordo entre as redes (a incerteza "epistêmica"): se cada rede está segura, mas elas apontam classes diferentes, IM é alta.

5.9 "Conheço isto?": detecção fora da distribuição

Uma rede de classificação sempre responde uma das classes que conhece, mesmo diante de algo que nunca viu. Para saber quando não confiar, usam-se medidores de "fora da distribuição" (OOD). O **escore de energia** [35] usa os logits:

$$E(\mathbf{x}) = -T \ln \sum_{c=1}^{C} e^{z_c(\mathbf{x})/T} \tag{36}$$

Logits altos (rede segura) dão energia baixa; imagens estranhas tendem a dar energia alta. A **distância de Mahalanobis** [36] olha para o "espaço interno" da rede:

$$D(\mathbf{x}) = \min_c (\mathbf{g}(\mathbf{x}) - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{g}(\mathbf{x}) - \boldsymbol{\mu}_c) \tag{37}$$

$\mathbf{g}(\mathbf{x})$ é o vetor de 512 características que sai do GAP; $\boldsymbol{\mu}_c$ é a média das imagens de treino da classe c; e $\boldsymbol{\Sigma}$ é a covariância compartilhada, estimada com o encolhimento de Ledoit–Wolf [37]. D mede o quão longe a imagem está da classe mais próxima.

Cada medidor tem um limiar: o quantil 99% (q99) dos escores das imagens de validação. Por construção, só 1% das imagens normais passa dele. No estudo de caso, a referência é ajustada no treino e calibrada na validação, separadamente para cada uma das 5 redes; no teste do Gravity Spy, 0,4%–1,2% das imagens ficam acima do q99 de Mahalanobis, como esperado. A decisão final tem três níveis:

- **rejeitar (fora do domínio):** pelo menos 3 das 5 redes acima do q99 de energia, ou pelo menos 3 acima do q99 de Mahalanobis;
- **revisar:** sem rejeição, mas pelo menos 3 redes acima do q95 de algum medidor, ou menos de 80% dos votos na classe vencedora, ou informação mútua acima de 0,10;
- **candidato em domínio:** nenhum dos casos anteriores. Só então a classe é relatada.

5.10 Interpretabilidade: Grad-CAM

O Grad-CAM [38] mostra onde a rede "olhou" para decidir. Ele pondera os mapas A^k de uma camada — no projeto, os 512 mapas de 9 × 11 posições da layer4 — pela média do gradiente do logit y^c da classe escolhida:

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{i,j}^k}, \qquad L^c = \mathrm{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{38}$$

Z é o número de posições do mapa. O mapa L^c é ampliado para o tamanho da imagem e sobreposto a ela. A resolução é baixa, e o mapa mostra onde estava a evidência usada — não prova que a decisão é confiável.

5.11 Quadro-resumo das ferramentas

Tabela 6 – Ferramentas de machine learning e deep learning usadas no estudo de caso, e para quê

Ferramenta ou técnica	Tipo	Uso no projeto
Regressão logística (scikit-learn)	ML clássico	linha de base sobre pixels
ResNet-18 (PyTorch, torchvision)	deep learning	classificador principal de 22 classes
Aprendizado por transferência	deep learning	começar com filtros pré-treinados no ImageNet
AdamW, entropia ponderada, parada antecipada	otimização	treino estável com classes desbalanceadas
Ensemble de 5 seeds	ML/DL	robustez e medida de desacordo (IM)
Escalonamento por temperatura	calibração	confiança que corresponde ao acerto
Energia e Mahalanobis com Ledoit–Wolf	detecção OOD	dizer "não conheço esta imagem"
Grad-CAM (autograd do PyTorch)	interpretabilidade	ver onde a rede olhou
Bootstrap, Theil–Sen, programação dinâmica	estatística	intervalos de confiança e ridge
NumPy, SciPy, gwpy, gwosc, h5py	processamento de sinais	dados, filtros, PSD e transformada Q

Versões: Python 3.11, PyTorch 2.5.1 com CUDA 12.1, torchvision 0.20.1, scikit-learn 1.9.1, gwpy 4.1.0, gwosc 0.8.3, NumPy 2.4.6, SciPy 1.17.1. Hardware: AMD Ryzen 5 5600G, 32 GB de RAM, NVIDIA RTX 3060 (12 GB); o treino das 5 seeds leva 1 h 46 min.

> **Em poucas palavras.** A rede é uma ResNet-18 de 11 milhões de parâmetros que transforma uma imagem de 140 × 170 em 22 probabilidades. Cinco cópias treinadas com seeds diferentes votam; dois medidores dizem se a imagem se parece com as de treino; e o Grad-CAM mostra onde a rede olhou.

6. Estudo de caso: evolução e resultados do Projeto 3

6.1 A evolução do projeto

O projeto passou por várias versões. Algumas conclusões iniciais foram revistas quando auditorias encontraram erros, e essas correções são parte importante do que o estudo de caso ensina (Tabela 7; as lições são discutidas na Seção 7.4).

Tabela 7 – Linha do tempo do projeto (2026)

Data	Marco	O que mudou
até set.	Versões v7 e v8 do pipeline	aquisição de 90 eventos do GWTC (230 canais aprovados); validação do GW150914 e primeiro treino da ResNet-18
27/09	Relatório V1	rede treinada (linha v9); Grad-CAM com a hipótese de “detecção por silhueta”; estudo de caso do GW151012
07/10	Relatório V2 e verificação do cálculo	descoberta de que os Q-grams do catálogo eram lidos transpostos (tempo e frequência trocados); estudo do GW151012 revogado
10/10	Correções do kit didático e proposta da linha 2.0	filtro casado e espectrograma do kit corrigidos; nova linha de código proposta
17/10	Errata do V2 e adoção da linha 2.0	painel vazio repetido 161 vezes; limiar OOD calibrado nas amostras do ajuste; modelos misturados no escore OOD
17–20/10	Relatório V3 e Relatório de Resultados	reprodução independente bit a bit; macro-F1 de 0,945; 475 de 475 sinais reais rejeitados
26/10	Plano 1: teste de fechamento	câmera não validada: macro-F1 0,714 e rejeição de 6,0%
27/10	Plano 1.1: receita do Gravity Spy	câmera não validada: macro-F1 0,752 e rejeição de 3,5%

6.2 Resultado nos glitches do Gravity Spy

Nas imagens do próprio Gravity Spy, a rede da linha 2.0 é muito boa (Tabela 8). A melhora em relação à linha v9 vem principalmente da limpeza dos dados.

Tabela 8 – Desempenho no conjunto de teste do Gravity Spy (média ± desvio das 5 seeds)

Métrica	Linha v9 (Relatório V2)	Linha 2.0 (atual)
Macro-F1	0,856	0,945 ± 0,009
Acurácia balanceada	0,844	0,946 ± 0,012
Kappa de Cohen	0,900	0,961 ± 0,004
Acurácia	0,909	0,965 ± 0,004
Regressão logística (macro-F1)	–	0,797

As classes mais difíceis continuam sendo as “genéricas”, No_Glitch (F1 de 0,76) e None_of_the_Above (0,77), que por definição reúnem imagens variadas (Figuras 9 e 10).

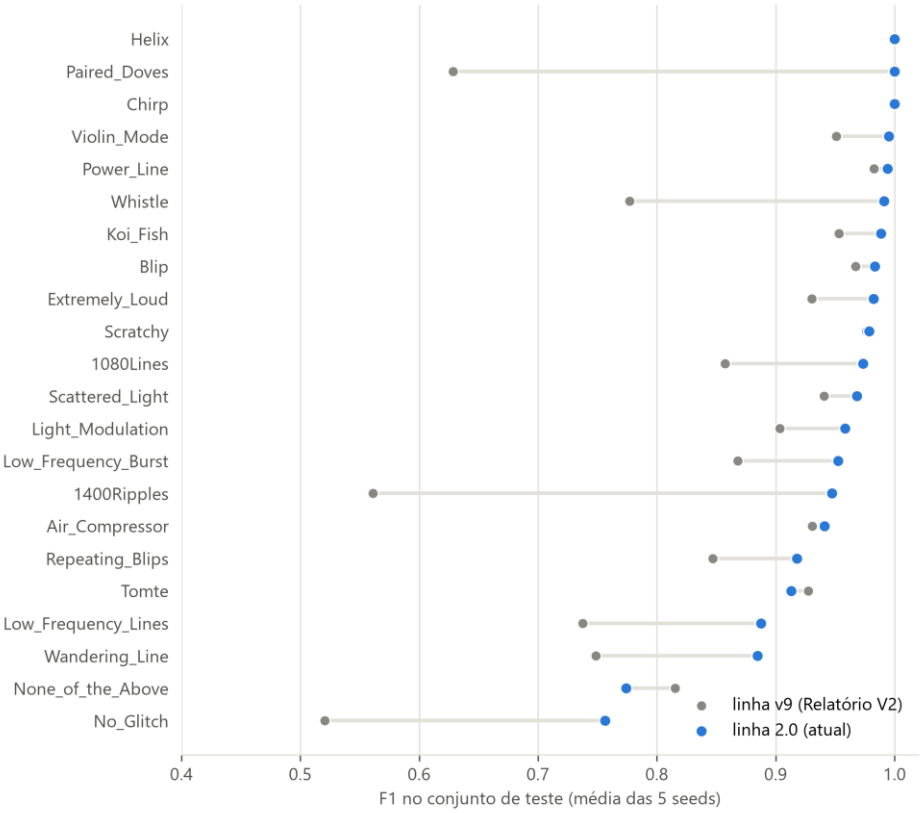


Figura 9 – F1 por classe, linha v9 × linha 2.0 (média das 5 seeds no conjunto de teste). O teste da v9 incluía 21 cópias do painel vazio; o da 2.0, não.

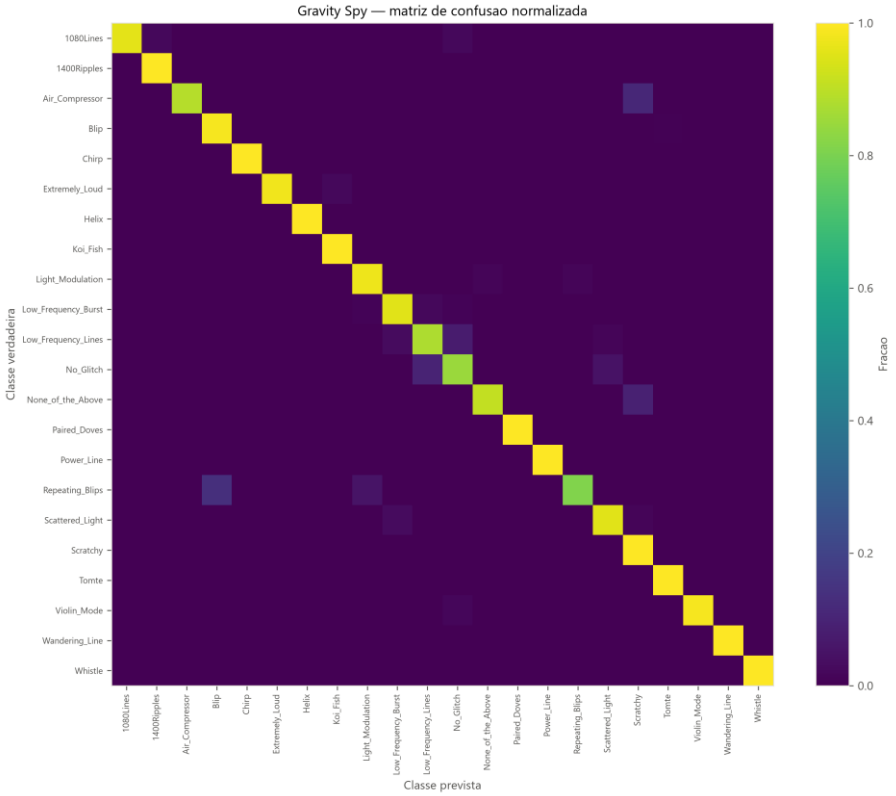


Figura 10 – Matriz de confusão da seed 42 no teste do Gravity Spy. Cada linha mostra, para uma classe verdadeira, a fração das imagens atribuída a cada classe. A diagonal quase completa indica poucos erros.

6.3 Resultado nos eventos reais do GWTC-4

Com a renderização por percentis, os 475 sinais dos 218 eventos ficam acima do limiar de Mahalanobis nas 5 redes: 100% rejeitados. Se forçarmos uma classe, a rede distribui quase tudo entre None_of_the_Above (243) e Extremely_Loud (231), inclusive o GW150914. Uma auditoria mostra que essas respostas dependem de propriedades globais da imagem: nove estatísticas simples de brilho preveem se a rede dirá None_of_the_Above com acurácia balanceada de 0,769 (AUROC de 0,817) — comportamento típico de um classificador fora do seu domínio, que responde a "atalhos" da imagem.

Tabela 9 – Os mesmos 475 sinais, modelos e limiares; muda só a renderização

Resultado	Percentis	Escala Gravity Spy
Rejeitados (fora do domínio)	475	253
Em revisão	0	205
Candidatos em domínio	0	17
Acima do q99 de Mahalanobis	475	49
Acima do q99 de energia	0	228
Confiança média do ensemble	0,735	0,536
Sinais classificados como Chirp	0	58 (em 46 eventos)

Na escala Gravity Spy, a distância de Mahalanobis cai para perto do limiar, mas o escore de energia passa a rejeitar 228 sinais, porque a rede fica insegura: os dois medidores discordam. A fração de sinais chamados de Chirp cresce com a força do evento (correlação de Spearman de +0,50 com a SNR de rede), mas a maioria desses casos continua rejeitada, e os 17 candidatos em domínio são todos No_Glitch (Figuras 11 e 12).

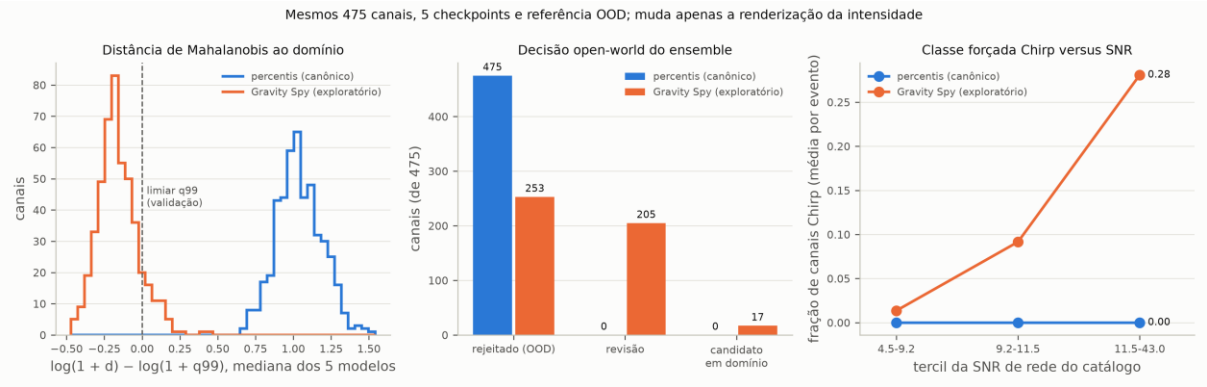


Figura 11 – Efeito da renderização nos 475 canais. À esquerda, o excesso de Mahalanobis passa de todo acima do limiar q99 para, na mediana, abaixo dele. No centro, as decisões. À direita, a fração de canais chamados de Chirp cresce com a SNR de rede, o que só acontece na escala Gravity Spy.

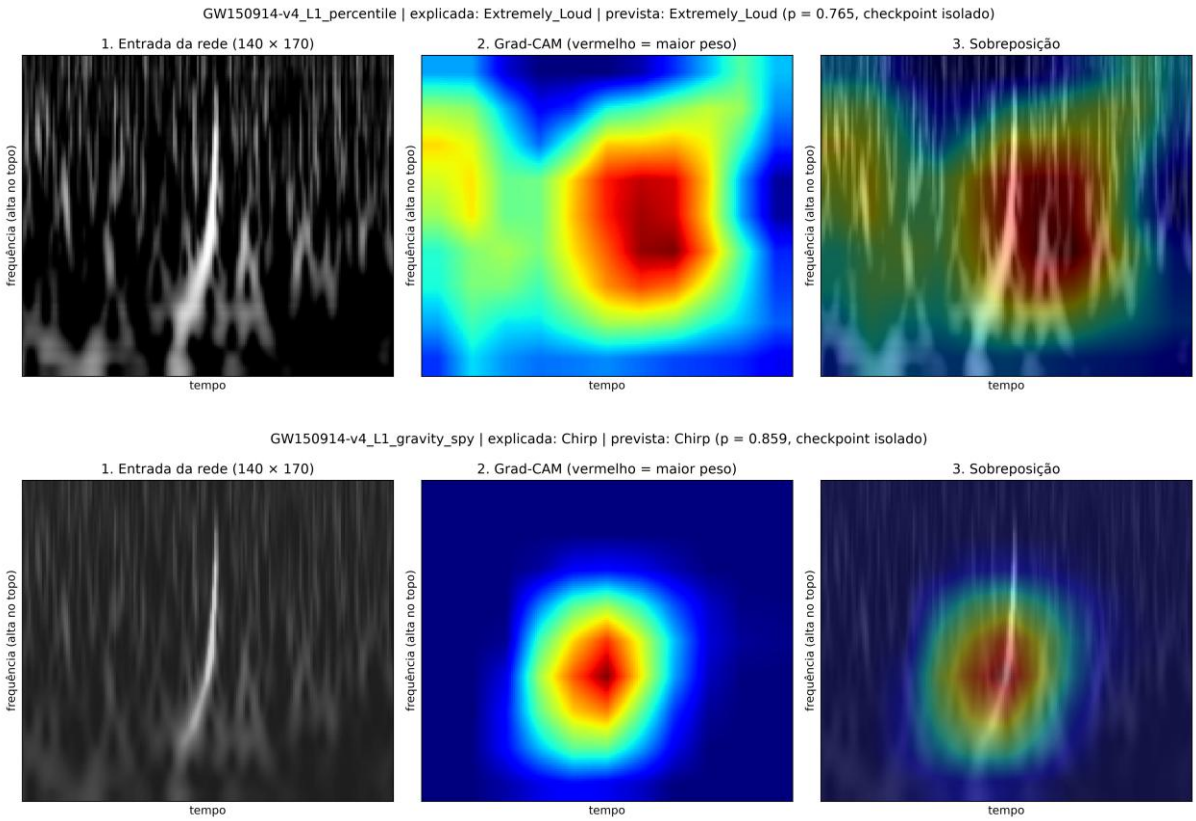


Figura 12 – Grad-CAM no GW150914 (L1, seed 42). Em cima, renderização por percentis: a rede responde Extremely_Loud. Embaixo, escala Gravity Spy: responde Chirp (p = 0,86), com a atenção máxima sobre a trilha do sinal.

6.4 Isolando a câmera: o teste de fechamento

Nos eventos, duas coisas mudam ao mesmo tempo: a câmera (Gravity Spy → nosso pipeline) e o conteúdo (glitches → ondas gravitacionais). O teste de fechamento separa as duas: baixamos do GWOSC o strain público nos horários de glitches que o Gravity Spy já rotulou (corrida O1, 2015–2016), geramos as imagens com a nossa câmera e verificamos se a rede os reconhece. Como a classe verdadeira é conhecida, o teste diz se a câmera funciona. A Tabela 10 compara as duas câmeras.

Tabela 10 – As duas câmeras: Gravity Spy (imagens de treino) e o nosso pipeline (imagens dos eventos)

Etapa	Gravity Spy	Nosso pipeline
Dados	64 s a 16 kHz (h(t) online)	32 s a 4 kHz (GWOSC, recalibrado)
Transformada Q	Q de 4 a 64; busca em 10–2.048 Hz; frequência linear	Q de 4 a 64; 20–512 Hz; frequência logarítmica
Janela da imagem	1 s (uma das quatro vistas)	4 s, recortado no segundo central
Intensidade	energia de 0 a 25,5 em viridis, convertida para cinza	percentis ou escala Gravity Spy
Centro	pico do gatilho Omicron	instante GPS do evento

O conjunto de fechamento tem 150 arquivos de O1 (4.096 s cada, até ~19,5 GB baixados e apagados após o uso) e 983 glitches reconstruídos. Só entram glitches em trechos com dados públicos: 1.404 glitches de O1 caem fora deles. Decidem os 451 glitches dos conjuntos de validação e teste (17 classes com pelo menos 10 exemplos), depois de excluir 17 imagens inválidas. Os critérios foram fixados antes de rodar (Tabela 11).

Tabela 11 – Critérios preregistrados (configs/plano1.toml)

Critério	O que mede	Limiar
C0	macro-F1 dos modelos 20–512 Hz no teste oficial / cauda OOD no teste	≥ 0,85 / ≤ 0,02
C1	macro-F1 no conjunto de fechamento (classes com ≥ 10 exemplos)	≥ 0,85
C1	acurácia no conjunto de fechamento	≥ 0,85
C2	fração rejeitada como fora do domínio	≤ 0,05

Foram testadas quatro condições, combinando os modelos atuais (painel de 10–2.048 Hz) ou modelos retreinados com os painéis recortados em 20–512 Hz (a mesma faixa dos eventos; macro-F1 de 0,854 no teste oficial, C0 atendido) com as duas renderizações:

Tabela 12 – Resultado do teste de fechamento, versão 1.0 (451 glitches; intervalos de 95%)

Condição	Acurácia	Macro-F1 [IC 95%]	Rejeição OOD [IC 95%]	Veredito
A1 · atuais + percentis	0,086	0,043 [0,023; 0,059]	1,000 [1,00; 1,00]	não passou
A2 · atuais + escala GS	0,659	0,642 [0,585; 0,676]	0,341 [0,283; 0,406]	não passou
B1 · 20–512 Hz + escala GS	**0,716**	**0,714 [0,660; 0,747]**	**0,060 [0,032; 0,094]**	**não passou**
B2 · 20–512 Hz + percentis	0,060	0,022 [0,014; 0,030]	1,000 [1,00; 1,00]	não passou

GS: Gravity Spy. B1 é a condição que decide; as outras são comparação.

Três leituras: (i) a renderização por percentis é incompatível com a rede — mesmo glitches conhecidos são 100% rejeitados (A1 e B2), forte evidência de que a rejeição total dos eventos vinha, em boa parte, da câmera; (ii) igualar a intensidade (A1 → A2) leva o macro-F1 de 0,04 a 0,64, e igualar também a faixa de frequência (A2 → B1) leva a 0,71 e derruba a rejeição de 34% para 6%; (iii) mesmo assim, B1 não atinge os critérios. Os glitches curtos são reconhecidos quase como no painel oficial — Blip 0,94, Koi_Fish 0,96, Tomte 0,97, Chirp 0,97 —, mas as estruturas longas falham: Low_Frequency_Lines 0,05, No_Glitch 0,21 e Whistle 0,51, quase sempre confundidas com Violin_Mode (Figuras 13 e 14).

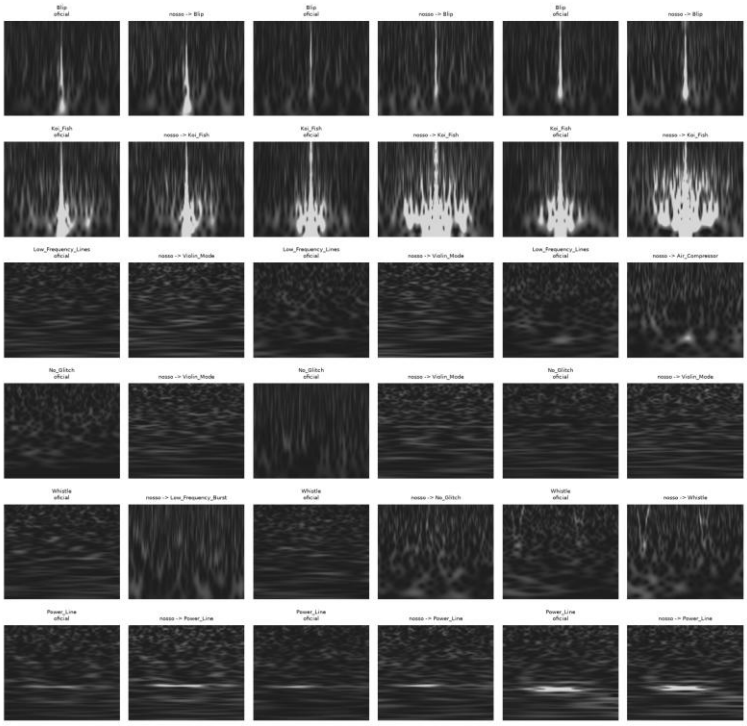


Figura 13 – Painel oficial recortado em 20–512 Hz (esquerda de cada par) e a imagem da nossa cadeia na versão 1.0 (direita), com a classe dada pelo ensemble. Blip, Koi_Fish e Power_Line coincidem; em Low_Frequency_Lines e No_Glitch, a nossa imagem tem listras horizontais.

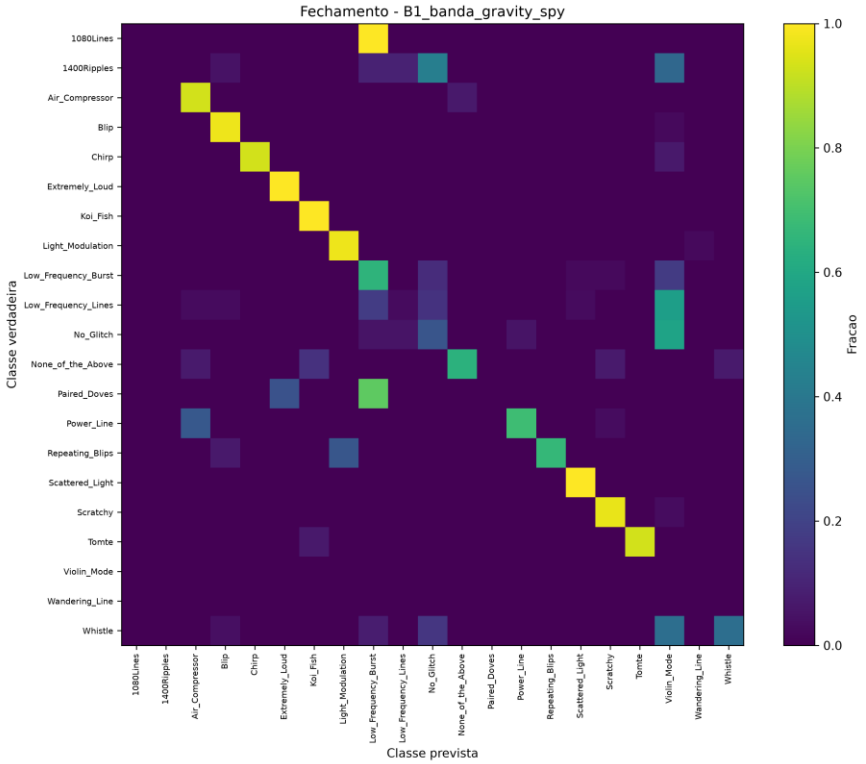


Figura 14 – Matriz de confusão normalizada da condição B1 (versão 1.0) nos 451 glitches de avaliação. As classes curtas ficam na diagonal; Low_Frequency_Lines, No_Glitch e Whistle migram para Violin_Mode.

6.5 A receita do Gravity Spy

A investigação apontou o modo de calcular a transformada Q. O Gravity Spy procura o Q em 10–2.048 Hz, com frequência linear e resoluções fixas (0,002 s no tempo e 0,5 Hz na frequência), sobre 64 s de

dados a 16 kHz [39]; a nossa cadeia calculava direto em 20–512 Hz, com frequência logarítmica, sobre 32 s a 4 kHz. Um piloto com 82 glitches do conjunto de **treino** — para não escolher a câmera olhando o conjunto que decide — comparou quatro variantes pela correlação mediana de pixels com o painel oficial (Tabela 13).

Tabela 13 – Piloto da versão 1.1: correlação mediana de pixels com o painel oficial

Classe	Receita 2.0 4 kHz, 32 s	Gravity Spy 4 kHz, 32 s	Gravity Spy 4 kHz, 64 s	Gravity Spy 16 kHz, 64 s
Blip	0,859	0,900	0,907	0,907
Koi_Fish	0,888	0,897	0,906	0,907
Tomte	0,907	0,921	0,924	0,924
Whistle	0,647	0,719	0,710	0,709
Low_Frequency_Lines	0,603	0,604	0,607	0,607
Power_Line	0,609	0,617	0,612	0,612
Scattered_Light	0,457	0,470	0,473	0,473
No_Glitch	0,601	0,602	0,434	0,434
Média	**0,697**	**0,716**	**0,697**	**0,697**

Venceu a receita do Gravity Spy a 4 kHz com 32 s. Janelas de 64 s pioraram o No_Glitch, e 16 kHz não trouxe ganho.

Com a variante escolhida, o conjunto de fechamento foi refeito e avaliado de novo, sem retreinar a rede (Tabela 14).

Tabela 14 – Resultado do teste de fechamento, versão 1.1 (451 glitches; intervalos de 95%)

Condição	Acurácia	Macro-F1 [IC 95%]	Rejeição OOD [IC 95%]	Veredito
B1 v1.0 (referência)	0,716	0,714 [0,660; 0,747]	0,060 [0,032; 0,094]	não passou
B1 v1.1 (decide)	**0,747**	**0,752 [0,696; 0,780]**	**0,035 [0,014; 0,060]**	**não passou**
A2 v1.1 (comparação)	0,681	0,668 [0,606; 0,703]	0,313 [0,257; 0,370]	não passou

A rejeição passou a atender o critério C2 (3,5%), mas o macro-F1 ficou em 0,752, e nem o limite superior do intervalo (0,78) chega a 0,85. A receita explica parte da diferença: Low_Frequency_Burst sobe de 0,64 para 0,76, e None_of_the_Above, de 0,75 para 0,92. As estruturas longas, porém, não melhoram (Figuras 15 e 16).

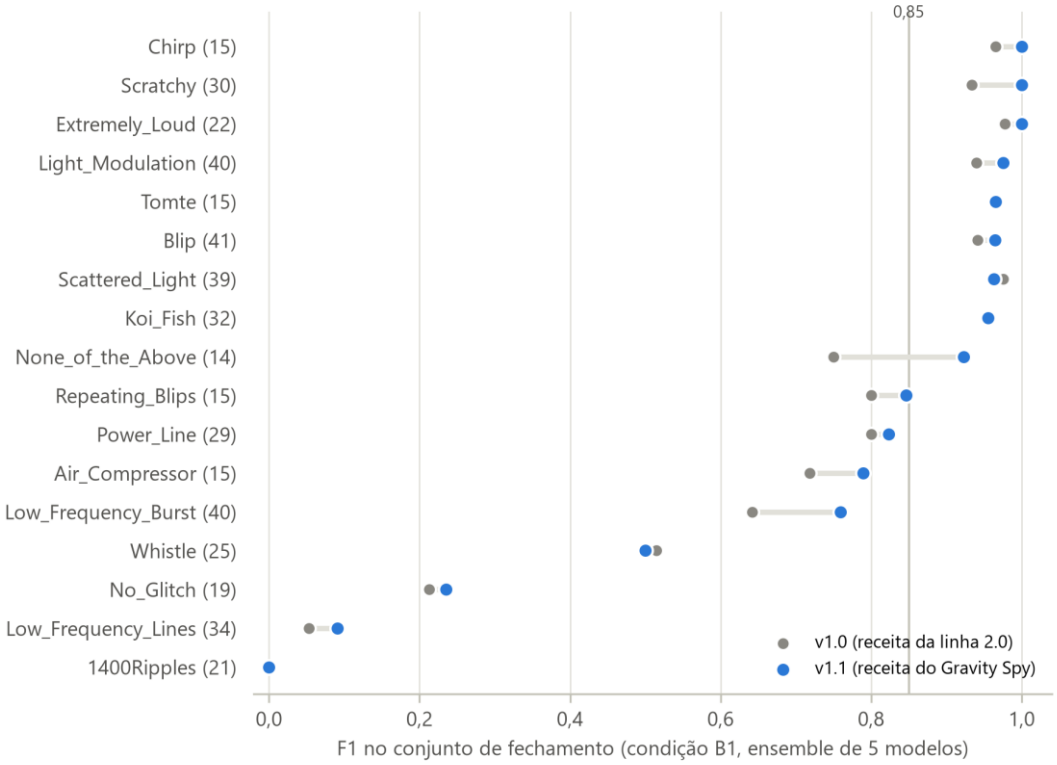


Figura 15 – F1 por classe na condição B1, antes (v1.0) e depois (v1.1) da receita do Gravity Spy, nas classes com pelo menos 10 exemplos (entre parênteses). Os glitches curtos ficam acima de 0,95; Low_Frequency_Lines, No_Glitch e Whistle continuam baixos; 1400Ripples fica fora da faixa de 20–512 Hz.

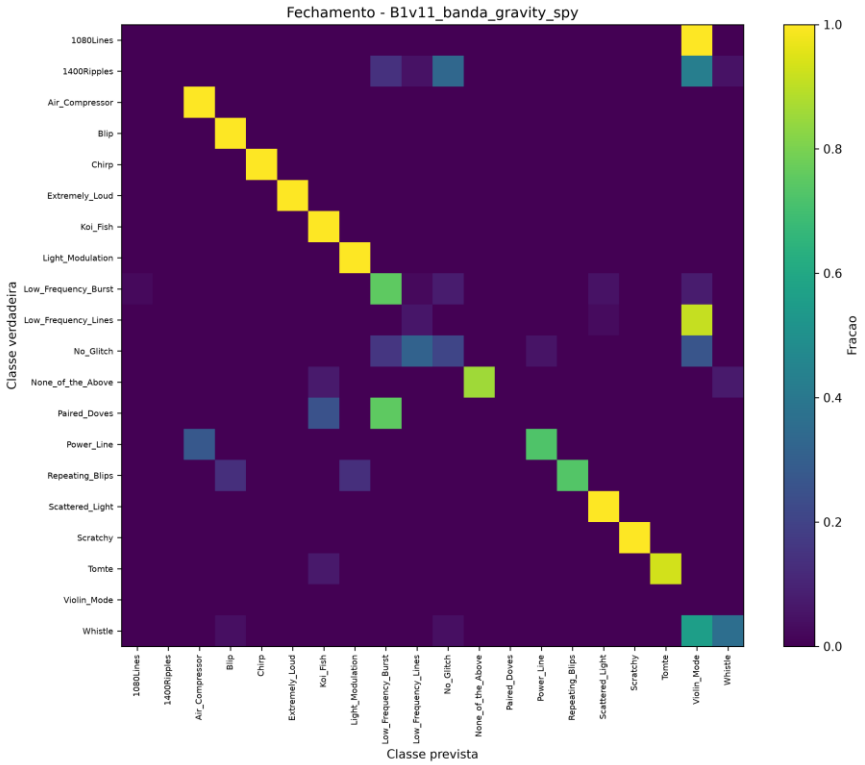


Figura 16 – Matriz de confusão normalizada da condição B1 na versão 1.1. Low_Frequency_Lines vai quase toda para Violin_Mode; Whistle se divide entre Violin_Mode e Whistle; No_Glitch se espalha entre Low_Frequency_Lines, No_Glitch e Violin_Mode.

6.6 Síntese dos resultados

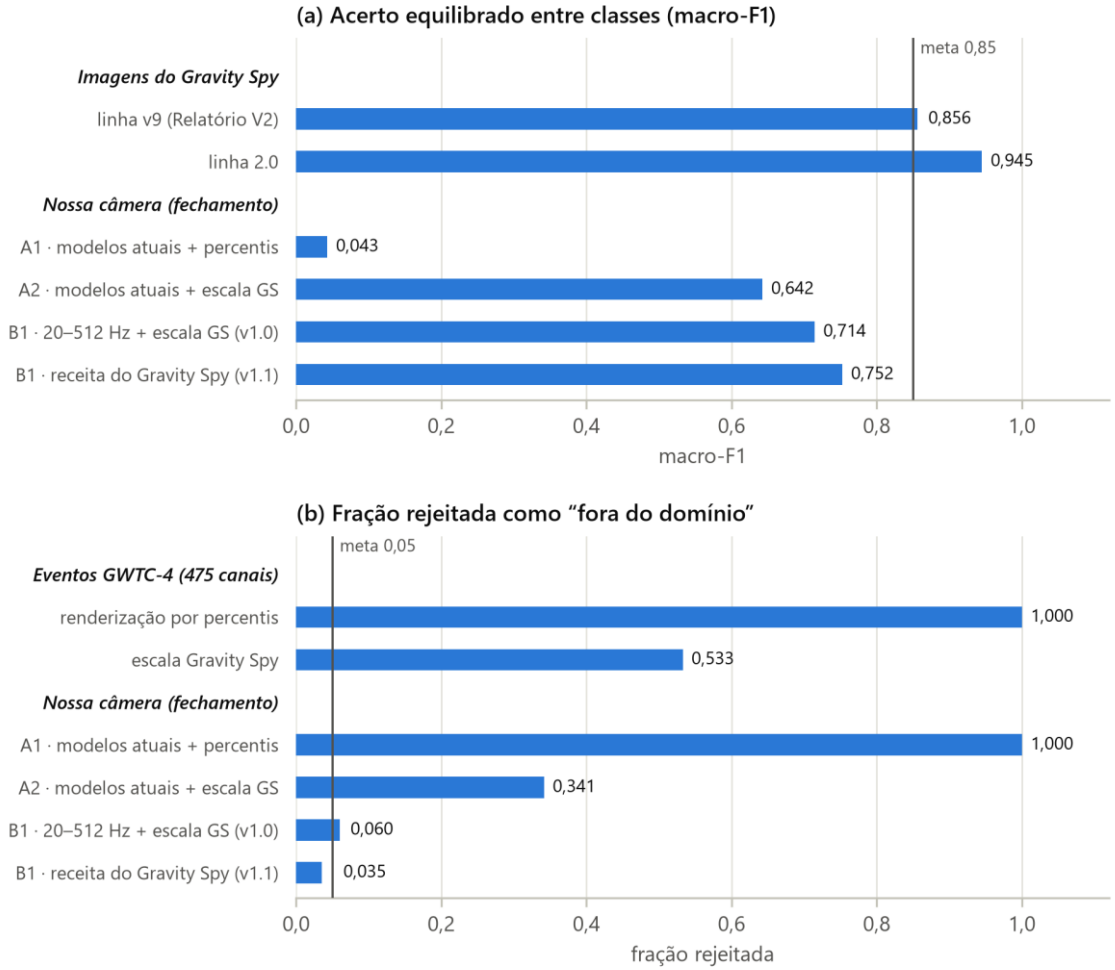


Figura 17 – Resumo da evolução. (a) Macro-F1: nas imagens do Gravity Spy, a rede passa da meta; com a nossa câmera, melhora a cada versão, mas fica abaixo de 0,85. (b) Fração rejeitada como fora do domínio: dos 100% iniciais à meta de 5%, atingida na versão 1.1. Nos eventos, a fração da escala Gravity Spy é por canal (253 de 475).

Tabela 15 – Onde estamos: o mesmo classificador em três situações

Situação	Conteúdo	Câmera	Resultado
Teste original	glitches	Gravity Spy	acurácia 0,965; macro-F1 0,945
Fechamento v1.1	os mesmos tipos de glitch	nossa	acurácia 0,747; macro-F1 0,752; rejeição 3,5%
Eventos GWTC-4	ondas gravitacionais	nossa	100% rejeitados (percentis); 53% (escala GS)

A queda de 0,965 para 0,747 em acurácia, com o mesmo tipo de conteúdo, se deve só à troca de câmera.

> **Em poucas palavras.** A rede reconhece muito bem as imagens do Gravity Spy. Quando os mesmos glitches passam pela nossa câmera, ela ainda acerta os curtos, mas erra as linhas e o ruído de fundo. A forma de gerar as imagens explica boa parte da rejeição dos eventos reais.

7. Discussão e perspectivas

7.1 O principal obstáculo é a mudança de domínio

A rede aprendeu a aparência específica das imagens do Gravity Spy: fundo, contraste, faixa de frequência e textura do ruído. Qualquer mudança sistemática nessa aparência basta para tirá-la do domínio. O teste de fechamento mostrou isso com glitches de classe conhecida: só trocar a câmera derruba a acurácia de 0,965 para 0,72–0,75. Esse fenômeno, a **mudança de domínio** (*domain shift*), é comum em aprendizado de máquina e tem uma literatura própria de métodos de adaptação [40]. No contexto de ondas gravitacionais, ele aparece também quando redes treinadas em ruído simulado são aplicadas a ruído real [4, 20].

7.2 O que a câmera explica, e o que ainda não explica

Os glitches curtos e intensos (Blip, Koi_Fish, Tomte, Chirp, Extremely_Loud) são reconhecidos quase como no painel oficial. As falhas se concentram nas estruturas longas ou estacionárias — linhas e ruído de fundo —, que dependem mais do branqueamento, da calibração e do fundo da imagem. Com a nossa câmera, o fundo tem listras horizontais que a rede lê como uma linha (Violin_Mode).

Algumas hipóteses foram testadas e descartadas: o Q escolhido em outra faixa (o Q mediano é igual nas duas faixas), a janela de 64 s e a amostragem a 16 kHz. A explicação restante mais provável é a **calibração**: o Gravity Spy usou o h(t) "online" da época, enquanto o GWOSC publica a versão recalibrada, que não temos como reproduzir. Vale notar que a investigação que motivou a versão 1.1 olhou glitches de avaliação; o piloto feito só com o conjunto de treino reduz, mas não elimina, esse viés.

7.3 Classificar glitches não é detectar ondas

A classe Chirp do Gravity Spy descreve um formato, e só 60 dos 7.807 exemplos de treino são Chirp. Mesmo com a câmera validada, a rede continuaria sendo uma classificadora de glitches. Detectar ondas gravitacionais exige outra abordagem, bem estabelecida na literatura: redes treinadas com sinais simulados injetados em ruído [18, 19, 21] ou classificadores que separam sinal de glitch em candidatos reais [22].

7.4 Lições metodológicas

- **Conferir a orientação dos dados.** Um eixo trocado (tempo × frequência) produziu, no Relatório V1, conclusões que pareciam fortes — a "detecção por silhueta" e o estudo de caso do GW151012 — e que não se sustentaram.
- **Separar ajuste e calibração.** Calibrar o limiar OOD nas mesmas amostras usadas no ajuste subestimou as distâncias: 42,3% das imagens normais eram declaradas desconhecidas; com a separação, 1,4%.
- **Não misturar modelos.** Uma distância calculada no espaço interno de uma rede e comparada com a referência de outra não tem significado.
- **Limpar os dados de treino.** Um painel vazio repetido 161 vezes, com 8 rótulos diferentes, ensinou à rede que "imagem vazia é Extremely_Loud". A limpeza, junto com o treino determinístico, levou o macro-F1 de 0,856 para 0,945.
- **Fixar os critérios antes de rodar.** Nos testes de fechamento, os limiares foram definidos antes de qualquer resultado, o que torna os vereditos negativos interpretáveis.
- **Reproduzir.** A linha 2.0 foi refeita do zero em outra máquina; o treino saiu idêntico byte a byte.

Em poucas palavras. O projeto melhorou porque foi auditado. Cada erro encontrado — eixos trocados, dados duplicados, limiares mal calibrados — mudou uma conclusão, e o protocolo ficou mais rigoroso a cada versão.

7.5 Limitações

- O teste de fechamento usa só a corrida O1 (2015–2016); os eventos do catálogo vão de O1 a O4a.
- Violin_Mode, Helix e Wandering_Line não têm glitches de avaliação em trechos públicos de O1; Paired_Doves e 1080Lines têm menos de 10 e não entram no macro-F1.
- O Grad-CAM tem baixa resolução (9 × 11 posições) e foi examinado em poucos exemplos.
- Ridge e filtro casado são diagnósticos condicionados ao catálogo: não medem massa nem detectam eventos de forma independente.
- A comparação das renderizações nos eventos é exploratória; os testes de fechamento, por outro lado, tiveram critérios fixados antes de rodar.

7.6 Perspectivas

Treinar a rede com imagens da nossa câmera. Em vez de forçar a nossa câmera a imitar a do Gravity Spy, a rede aprende o domínio em que vai trabalhar. A opção mais barata é ajustar (*fine-tuning*) os modelos atuais com Q-grams gerados pelo nosso pipeline, mantendo os rótulos do Gravity Spy. O ponto de partida são os cerca de 980 glitches de O1 já reconstruídos (528 de treino e 451 de avaliação); para crescer, dá para usar O2 e O3, que também têm rótulos do Gravity Spy [9]. A validação seria o mesmo teste de fechamento, com os mesmos critérios, em glitches que a rede nunca viu.

Um detector com injeções, se o objetivo for encontrar eventos: injetar chirps simulados em ruído real do pipeline e treinar uma rede para separar sinal de glitch, na linha dos trabalhos revisados na Seção 5.1.

Pendências menores: entender a discordância entre os medidores de energia e de Mahalanobis; aplicar o pipeline a trechos sem evento (controles negativos); e decidir qual renderização vira a padrão.

Questões em aberto. 1. O objetivo final do projeto é classificar glitches ou detectar eventos? Isso decide entre as duas perspectivas acima.

2. Os limiares de 0,85 e 5% continuam valendo, ou faz sentido um critério por grupo de classes (curtas × longas)?

3. Há acesso a GPU ou cluster para ampliar os dados, ou o trabalho segue na máquina local?

8. Considerações finais

Este artigo revisou, em ordem, as etapas que levam do sinal de um detector de ondas gravitacionais à decisão de uma rede neural, usando o Projeto 3 como estudo de caso. Na primeira etapa, construímos a "câmera": o ruído é medido pela PSD, o sinal é branqueado e filtrado, e a transformada Q o transforma numa imagem que mostra claramente o chirp do GW150914. Na segunda, o pipeline recuperou a física do GW150914 de forma qualitativa — o atraso de ~7 ms e a inversão entre os detectores, a trilha do chirp, o template certo —, mas não as medidas quantitativas. Na terceira, uma ResNet-18 adaptada, treinada com cuidado (dados limpos, treino determinístico, ensemble, calibração e teste de fora da distribuição), classifica os glitches do Gravity Spy com macro-F1 de 0,945.

A rede, porém, não reconhece as imagens dos eventos reais. Os testes de fechamento mostraram por quê: a forma de gerar as imagens pesa muito. Com glitches de classe conhecida, só a troca de câmera reduz a acurácia de 0,965 para 0,75, e as classes de estrutura longa são as mais afetadas. Igualar a escala de intensidade, a faixa de frequência e a receita da transformada Q reduziu a rejeição de 100% para 3,5%, mas o acerto não chegou à meta.

O resultado negativo é, ele mesmo, uma contribuição: mostra que um classificador de glitches não pode ser transferido diretamente para os dados de outro pipeline, e mostra como medir isso — com critérios fixados antes, testes de fechamento e reprodução independente. Do ponto de vista didático, o percurso ilustra que cada etapa da cadeia — da janela de um filtro à escala de cinza de uma imagem — deixa marcas que uma rede neural aprende a reconhecer.

Agradecimentos

Este trabalho usou dados e software do Gravitational Wave Open Science Center (gwosc.org), serviço do LIGO Laboratory, da LIGO Scientific Collaboration, da Virgo Collaboration e da KAGRA, e as imagens e os rótulos do projeto Gravity Spy, construídos com a participação de voluntários da plataforma Zooniverse.

Referências

[1] B. P. Abbott et al. (LIGO Scientific Collaboration e Virgo Collaboration). Observation of gravitational waves from a binary black hole merger. *Physical Review Letters* 116, 061102 (2016).

[2] LIGO Scientific, Virgo e KAGRA Collaborations. GWTC-4.0: updating the Gravitational-Wave Transient Catalog with observations from the first part of the fourth LIGO-Virgo-KAGRA observing run. arXiv:2508.18082 (2025).

[3] B. P. Abbott et al. (LIGO Scientific Collaboration e Virgo Collaboration). A guide to LIGO–Virgo detector noise and extraction of transient gravitational-wave signals. *Classical and Quantum Gravity* 37, 055002 (2020).

[4] E. Cuoco et al. Enhancing gravitational-wave science with machine learning. *Machine Learning: Science and Technology* 2, 011002 (2021).

[5] B. S. Sathyaprakash e B. F. Schutz. Physics, astrophysics and cosmology with gravitational waves. *Living Reviews in Relativity* 12, 2 (2009).

[6] F. Robinet, N. Arnaud, N. Leroy, A. Lundgren, D. Macleod e J. McIver. Omicron: a tool to characterize transient noise in gravitational-wave detectors. *SoftwareX* 12, 100620 (2020).

[7] M. Zevin et al. Gravity Spy: integrating Advanced LIGO detector characterization, machine learning, and citizen science. *Classical and Quantum Gravity* 34, 064003 (2017).

[8] S. Bahaadini et al. Machine learning for Gravity Spy: glitch classification and dataset. *Information Sciences* 444, 172–186 (2018).

[9] J. Glanzer et al. Data quality up to the third observing run of Advanced LIGO: Gravity Spy glitch classifications. *Classical and Quantum Gravity* 40, 065004 (2023).

[10] R. Abbott et al. (LIGO Scientific Collaboration e Virgo Collaboration). Open data from the first and second observing runs of Advanced LIGO and Advanced Virgo. *SoftwareX* 13, 100658 (2021).

[11] P. D. Welch. The use of fast Fourier transform for the estimation of power spectra. *IEEE Transactions on Audio and Electroacoustics* 15, 70–73 (1967).

[12] S. Chatterji, L. Blackburn, G. Martin e E. Katsavounidis. Multiresolution techniques for the detection of gravitational-wave bursts. *Classical and Quantum Gravity* 21, S1809 (2004).

[13] D. M. Macleod et al. GWpy: a Python package for gravitational-wave astrophysics. *SoftwareX* 13, 100657 (2021).

[14] P. K. Sen. Estimates of the regression coefficient based on Kendall's tau. *Journal of the American Statistical Association* 63, 1379–1389 (1968).

[15] B. Allen et al. FINDCHIRP: an algorithm for detection of gravitational waves from inspiraling compact binaries. *Physical Review D* 85, 122006 (2012).

[16] A. Buonanno, B. R. Iyer, E. Ochsner, Y. Pan e B. S. Sathyaprakash. Comparison of post-Newtonian templates for compact binary inspiral signals in gravitational-wave detectors. *Physical Review D* 80, 084043 (2009).

[17] M. Razzano e E. Cuoco. Image-based deep learning for classification of noise transients in gravitational wave detectors. *Classical and Quantum Gravity* 35, 095016 (2018).

[18] D. George e E. A. Huerta. Deep neural networks to enable real-time multimessenger astrophysics. *Physical Review D* 97, 044039 (2018).

[19] H. Gabbard, M. Williams, F. Hayes e C. Messenger. Matching matched filtering with deep networks for gravitational-wave astronomy. *Physical Review Letters* 120, 141103 (2018).

[20] M. B. Schäfer, O. Zelenka, A. H. Nitz, F. Ohme e B. Brügmann. Training strategies for deep learning gravitational-wave searches. *Physical Review D* 105, 043002 (2022).

[21] M. B. Schäfer et al. First machine learning gravitational-wave search mock data challenge. *Physical Review D* 107, 023021 (2023).

[22] S. Alvarez-Lopez et al. GSpyNetTree: a signal-vs-glitch classifier for gravitational-wave event candidates. arXiv:2304.09977 (2023).

[23] I. Goodfellow, Y. Bengio e A. Courville. *Deep Learning*. MIT Press (2016).

[24] F. Pedregosa et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12, 2825–2830 (2011).

[25] S. Ioffe e C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. *ICML* (2015).

[26] K. He, X. Zhang, S. Ren e J. Sun. Deep residual learning for image recognition. *CVPR* (2016).

[27] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* (2019).

[28] J. Deng et al. ImageNet: a large-scale hierarchical image database. *CVPR* (2009).

[29] D. P. Kingma e J. Ba. Adam: a method for stochastic optimization. *ICLR* (2015).

[30] I. Loshchilov e F. Hutter. Decoupled weight decay regularization. *ICLR* (2019).

[31] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 37–46 (1960).

[32] C. Guo, G. Pleiss, Y. Sun e K. Q. Weinberger. On calibration of modern neural networks. *ICML* (2017).

[33] B. Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics* 7, 1–26 (1979).

[34] B. Lakshminarayanan, A. Pritzel e C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *NeurIPS* (2017).

[35] W. Liu, X. Wang, J. Owens e Y. Li. Energy-based out-of-distribution detection. *NeurIPS* (2020).

[36] K. Lee, K. Lee, H. Lee e J. Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. *NeurIPS* (2018).

[37] O. Ledoit e M. Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis* 88, 365–411 (2004).

[38] R. R. Selvaraju et al. Grad-CAM: visual explanations from deep networks via gradient-based localization. *ICCV* (2017).

[39] Projeto Gravity Spy. Código-fonte, função make_q_scans em gravityspy/utils/utils.py. github.com/Gravity-Spy/GravitySpy.

[40] Y. Ganin e V. Lempitsky. Unsupervised domain adaptation by backpropagation. *ICML* (2015).

Apêndice A — Glossário

Tabela 16 – Termos usados neste artigo

Termo	Significado
ASD / PSD	densidade espectral de amplitude / de potência: quanto ruído há em cada frequência
Branqueamento	divisão do espectro pelo ruído típico, para que todas as frequências pesem igual
Chirp	sinal cuja frequência e amplitude sobem com o tempo, típico da fusão de binárias
Ensemble	conjunto de redes treinadas de forma independente cujas respostas são combinadas
Época	uma passada completa da rede por todas as imagens de treino
Filtro casado	comparação do dado com um molde teórico da onda; dá a SNR
Glitch	ruído transitório do detector, de origem instrumental ou ambiental
GWOSC / GWTC	centro de dados abertos de ondas gravitacionais / catálogo oficial de eventos
Logit	número que a rede produz para cada classe antes do softmax
Macro-F1	média do F1 de todas as classes, com o mesmo peso para cada classe
Massa de chirp	combinação das duas massas que controla a evolução da frequência do chirp
Mudança de domínio	diferença entre os dados de treino e os dados em que o modelo é usado
OOD	fora da distribuição: entrada diferente das usadas no treino
Q-gram	imagem tempo × frequência produzida pela transformada Q
Seed	semente do gerador de números aleatórios; muda a inicialização e a ordem do treino
Softmax	função que transforma logits em probabilidades que somam 1
Strain	deformação relativa do braço do detector, h = ΔL / L
Teste de fechamento	passar dados de classe conhecida pela cadeia completa para verificar se ela funciona
Transferência de aprendizado	começar o treino a partir de uma rede já treinada em outra tarefa

Apêndice B — Reprodutibilidade

Todos os números do estudo de caso vêm de execuções registradas no repositório do projeto. As figuras novas deste artigo são geradas por `docs/relatorios/fontes/artigo_final/make_figures.py` (ambiente gwproject), e o documento por `build_docx.py`, na mesma pasta. Os principais arquivos são:

- `configs/processing.toml` — controle de qualidade, PSD, branqueamento, filtros, transformada Q e ridge;
- `configs/gravity_spy.toml` e `configs/research_program.toml` — treino, renderização e regras do OOD;
- `configs/plano1.toml` e `configs/plano1_v1_1.toml` — critérios preregistrados dos testes de fechamento;
- `src/gwproject/` — o pacote com todo o pipeline (preprocessing, gwosc_pipeline, gravity_spy, research);
- `docs/relatorios/` — os relatórios V1, V2 (e errata), V3, de Resultados e os Planos de Melhoria 1 e 1.1.